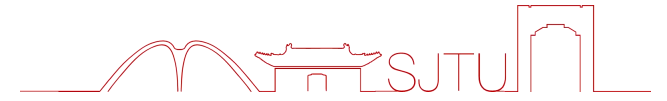




上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



Lecture 3 Introduction to Federated Learning

马汝辉

上海交通大学

饮水思源 · 爱国荣校



1

Introduction to Distributed Learning

2

Privacy and Security Issues in Deep Learning

3

Strengths of Federated learning

4

FedAvg Algorithm

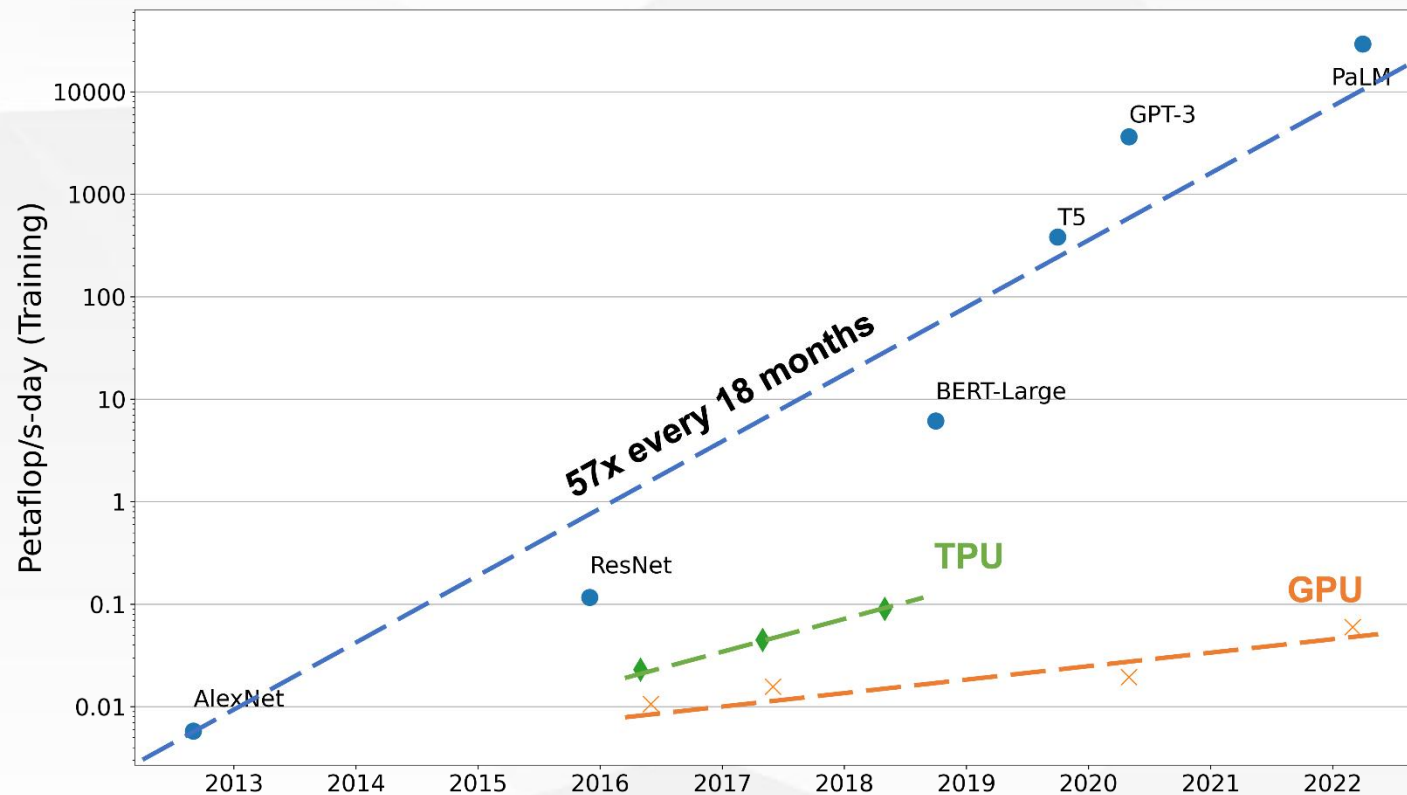


01

Introduction to Distributed Learning



Motivation



Huge difference in computing power and model size



Inefficiency of Memory

Parameter $1e9$

Feature Map $2e9$

Gradient $3e9$

Float32 4B

Total 24GB

NVIDIA 3070 8G

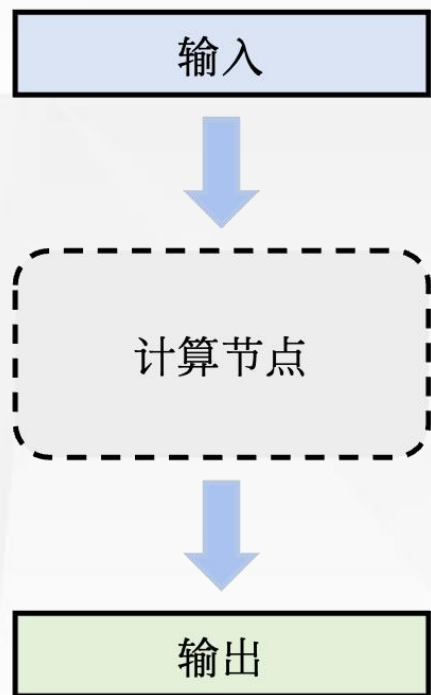
NVIDIA 3080 10G

NVIDIA 3090 24G

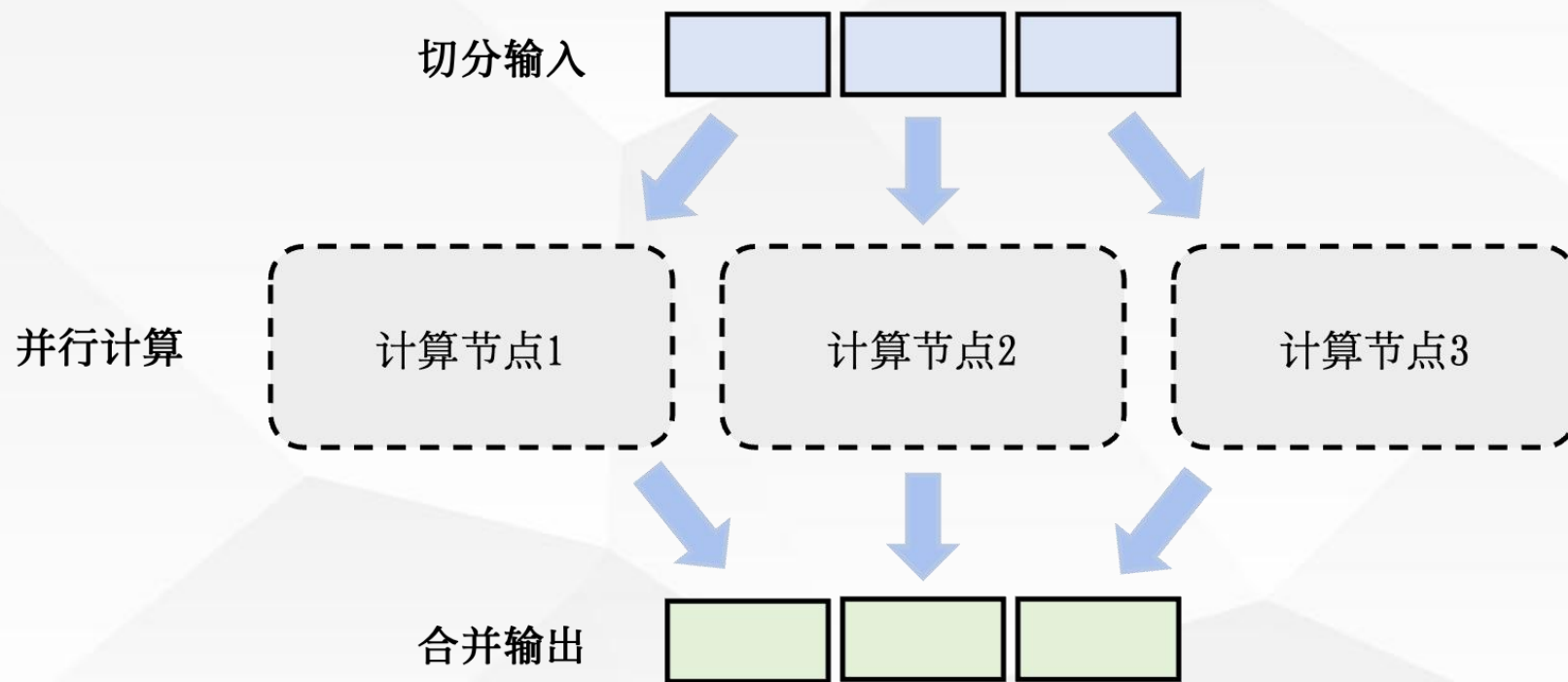
What if there is a bigger model?



Single Node vs Distributed



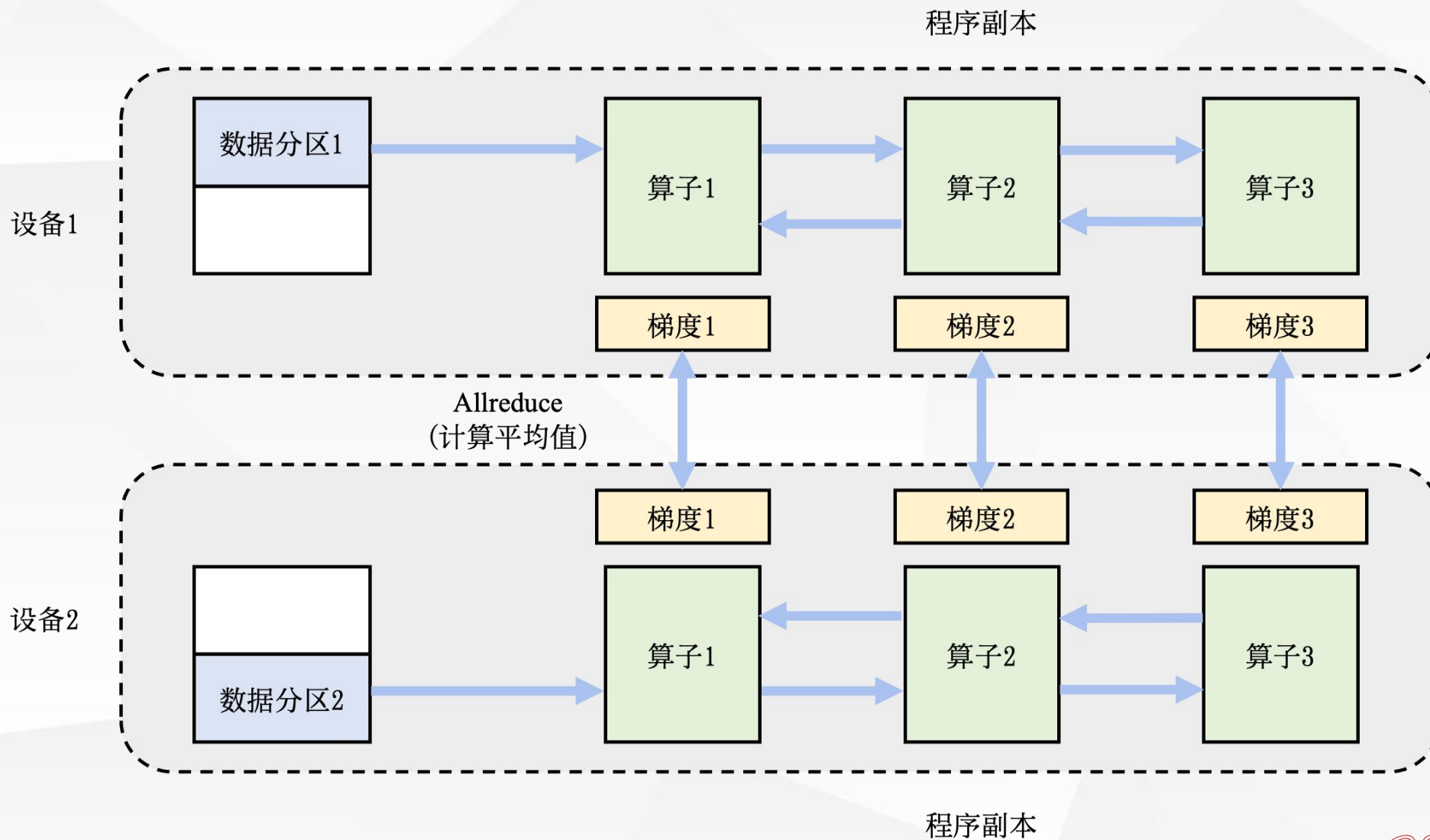
(a) 单节点执行



(b) 分布式多节点执行

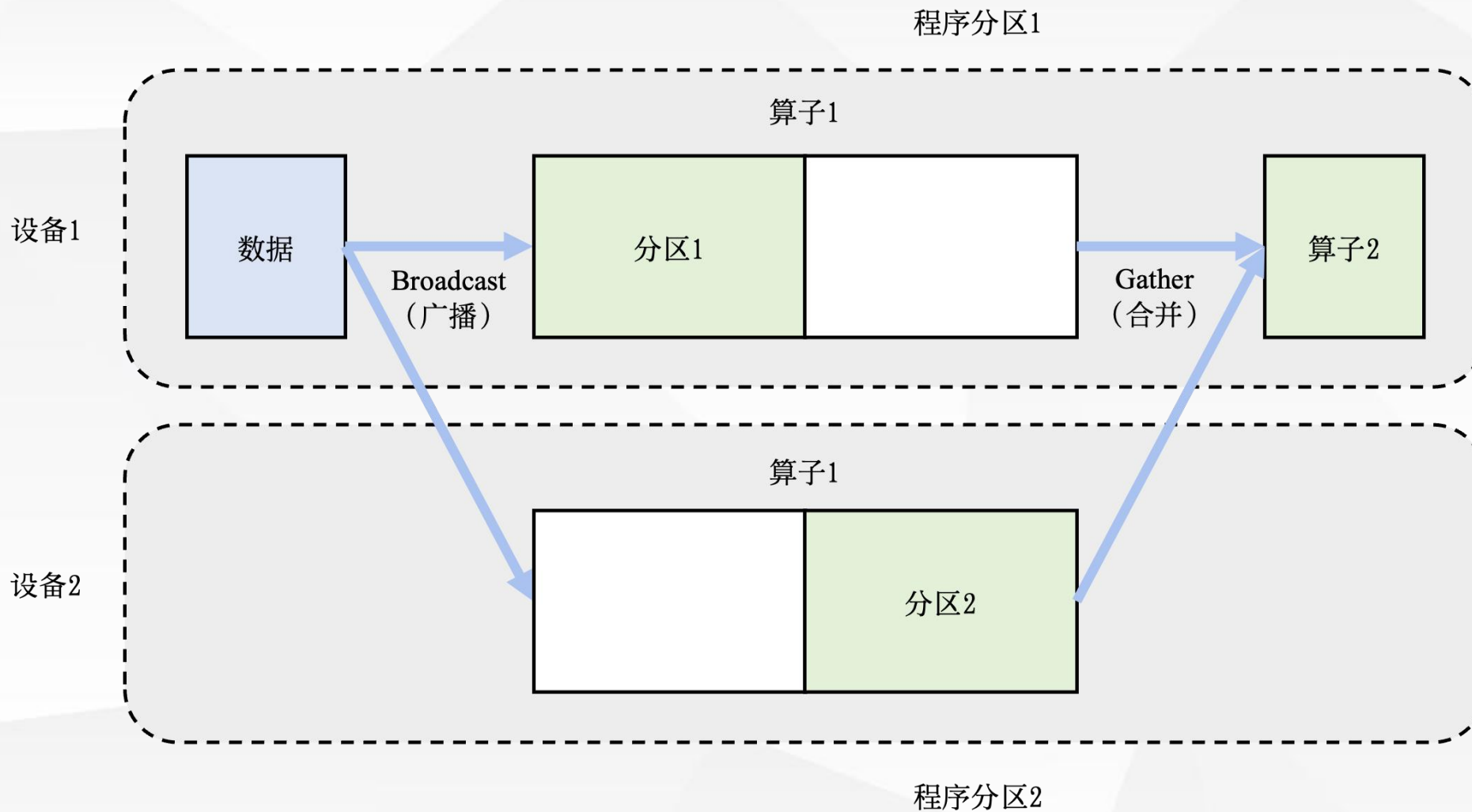


Distributed Methods#1



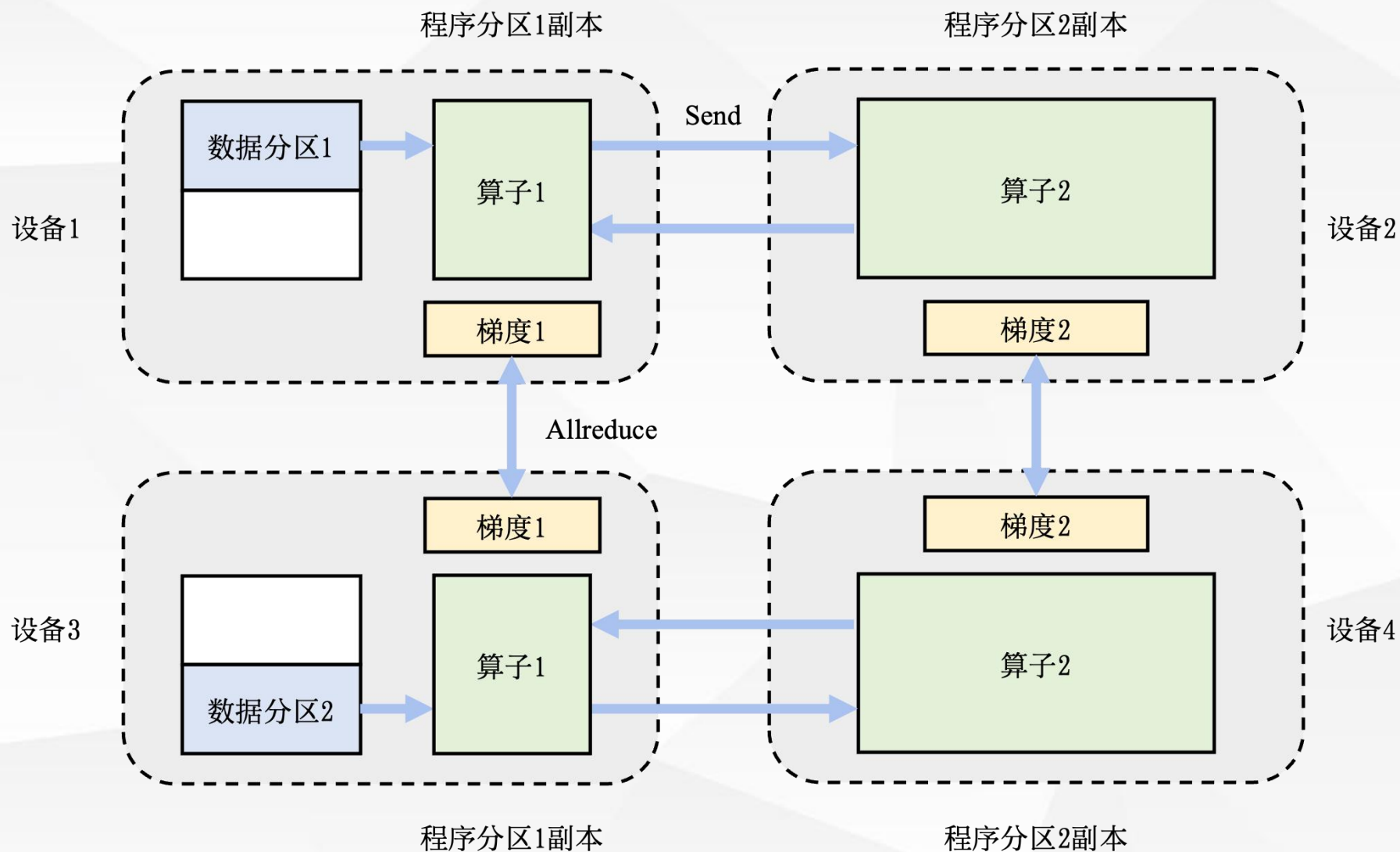


Distributed Methods#2



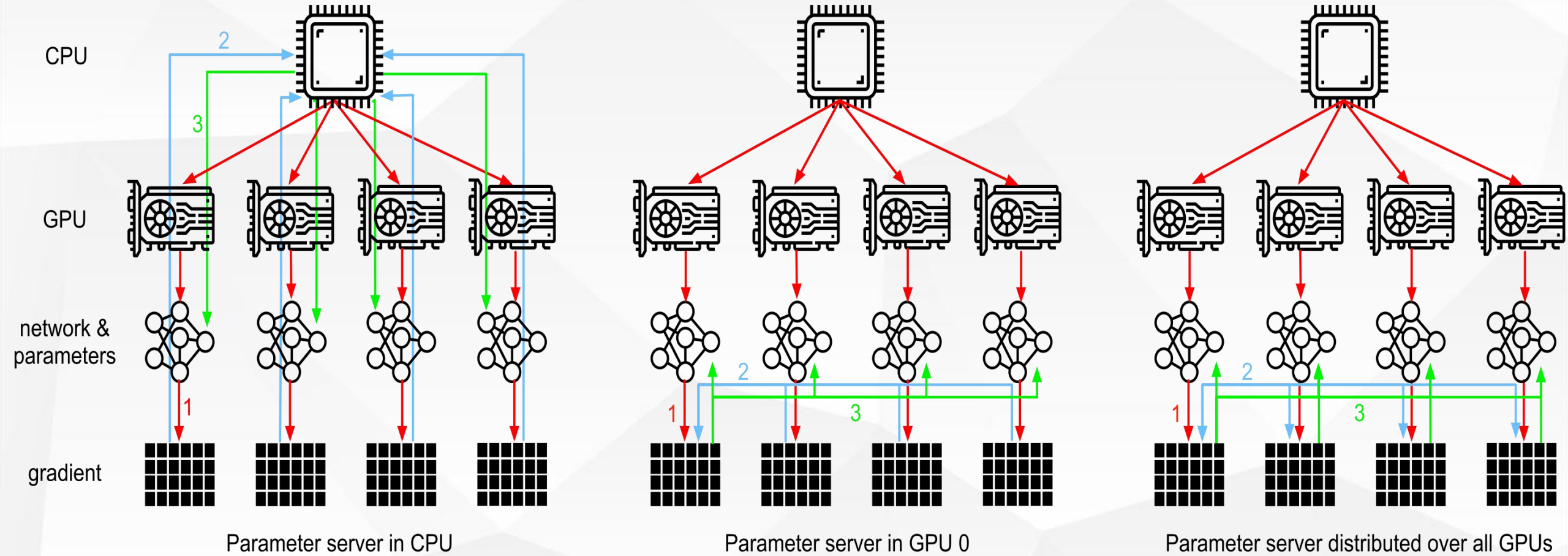


Distributed Methods#3





Problem Of Synchronization





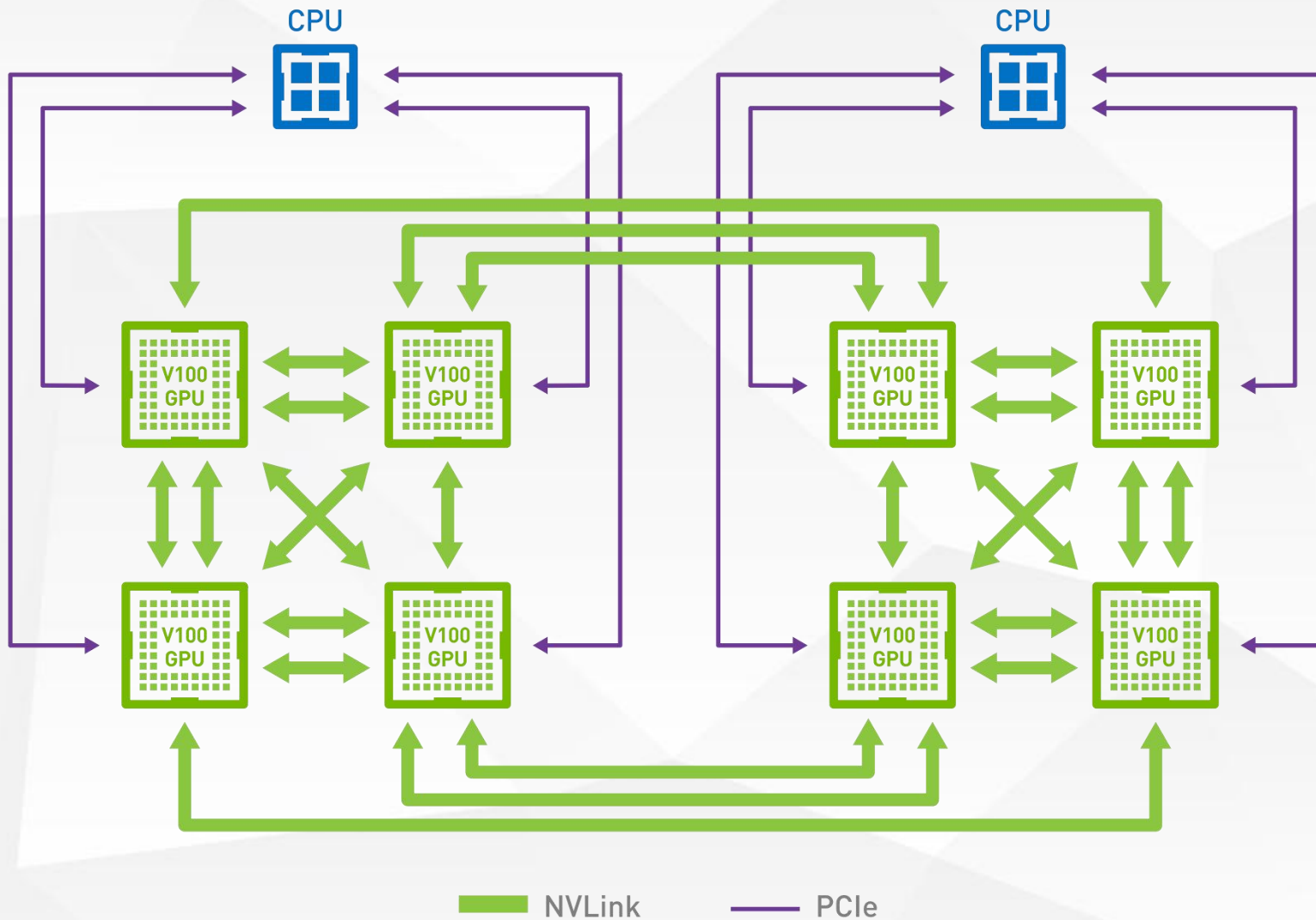
Problem Of Synchronization



- ⊗ Assume that the gradients are of 160 MB. In this case it takes 30 ms to send the gradients from all 3 remaining GPUs to the fourth one (each transfer takes $10 \text{ ms} = 160 \text{ MB} / 16 \text{ GB/s}$). Adding another 30 ms to transmit the weight vectors back we arrive at a total of **60 ms**.
- ⊗ If we send all data to the CPU we incur a penalty of 40 ms since *each* of the four GPUs needs to send the data to the CPU, yielding a total of **80 ms**.
- ⊗ we are able to split the gradients into 4 parts of 40 MB each. Now we can aggregate each of the parts on a different GPU *simultaneously* since the PCIe switch offers a full-bandwidth operation between all links. Instead of 30 ms this takes 7.5 ms, yielding a total of **15 ms** for a synchronization operation.



Ring Synchronization



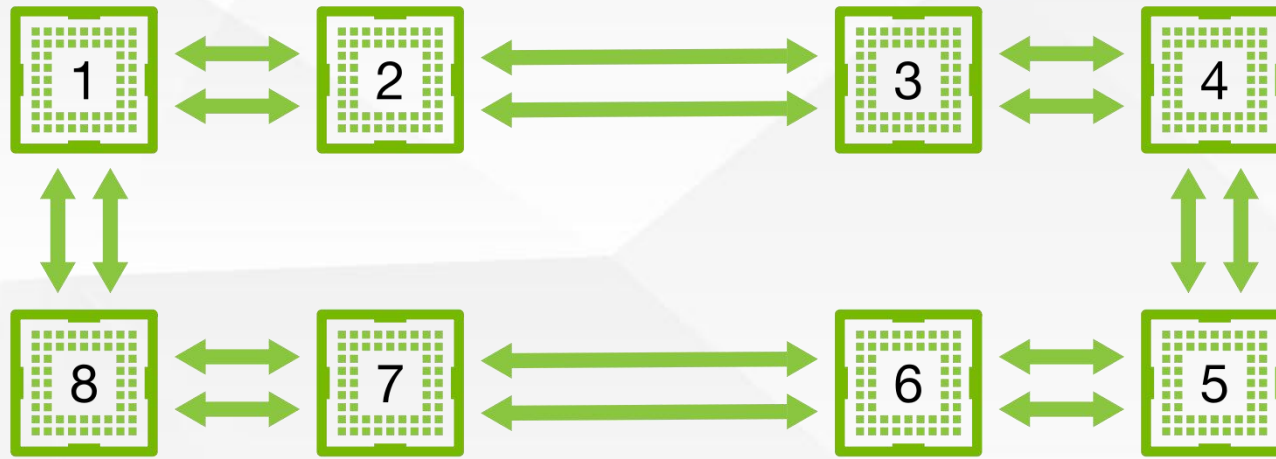
Each GPU connects to a host CPU via a PCIe link which operates at best at **16 GB/s**.

GPU also has 6 NVLink connections, each of which is capable of transferring **300 Gbit/s** bidirectionally.

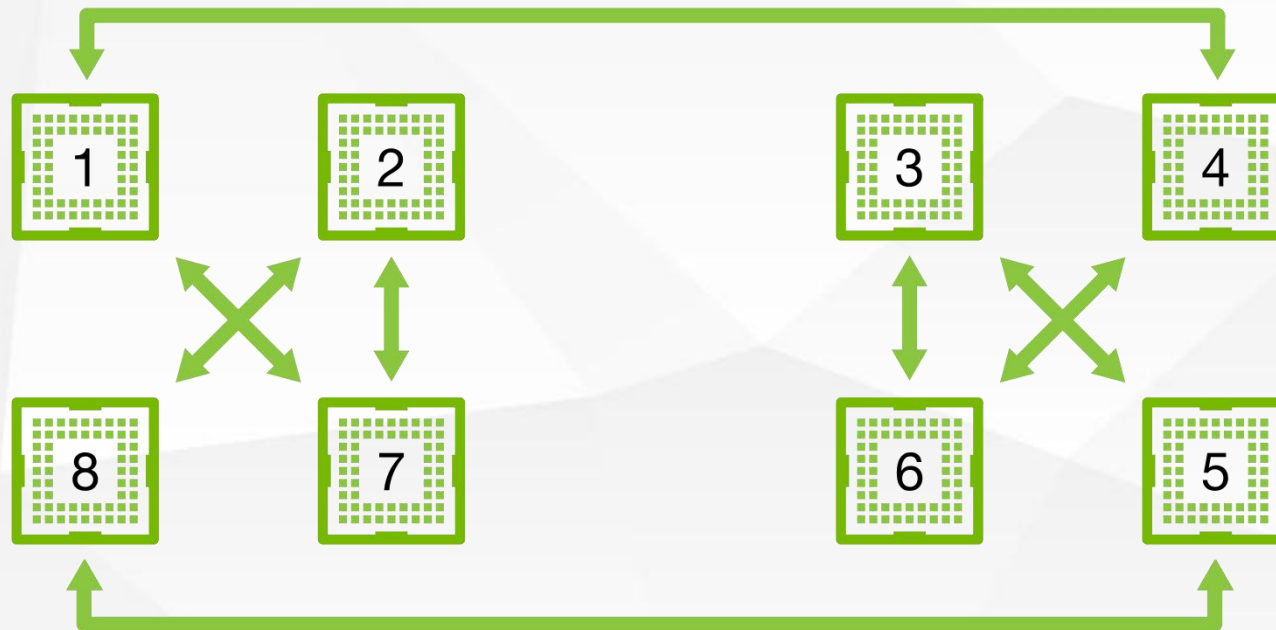
This amounts to around **18 GB/s per link**. In short, the aggregate NVLink bandwidth is significantly higher than the PCIe bandwidth. The question is how to use it most efficiently.



Ring Synchronization

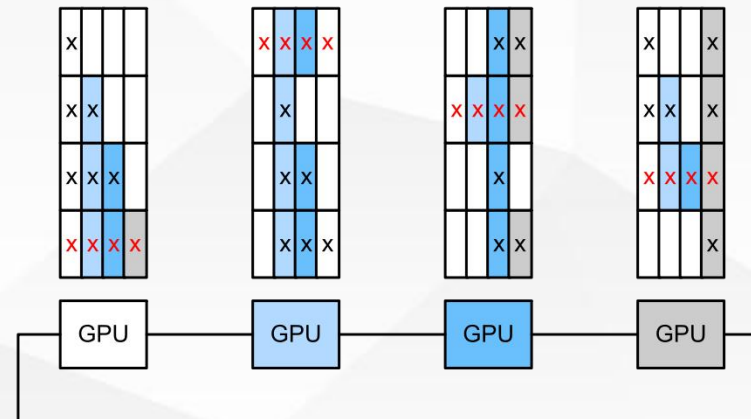
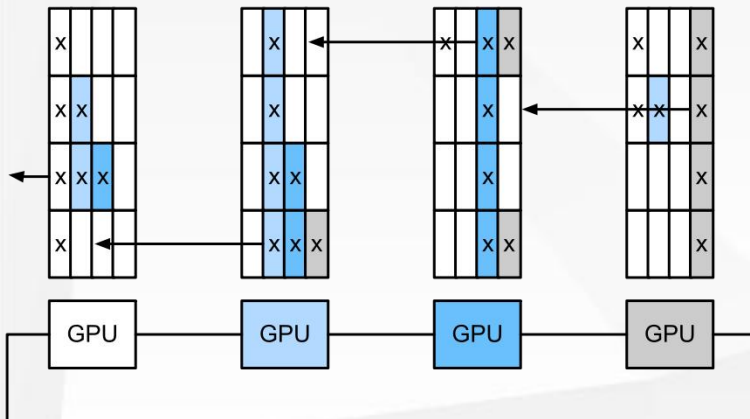
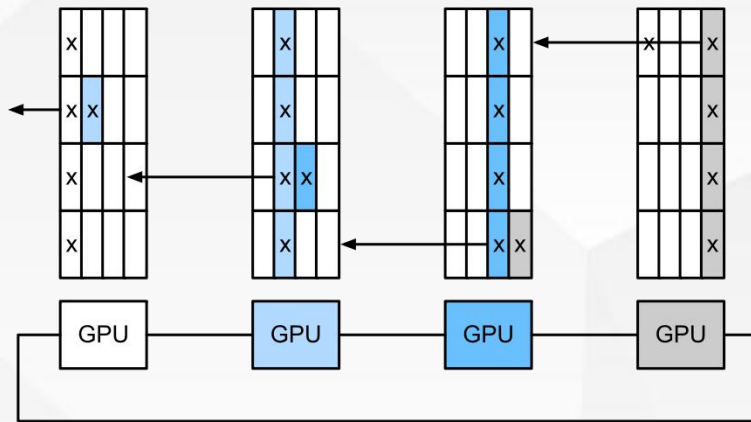
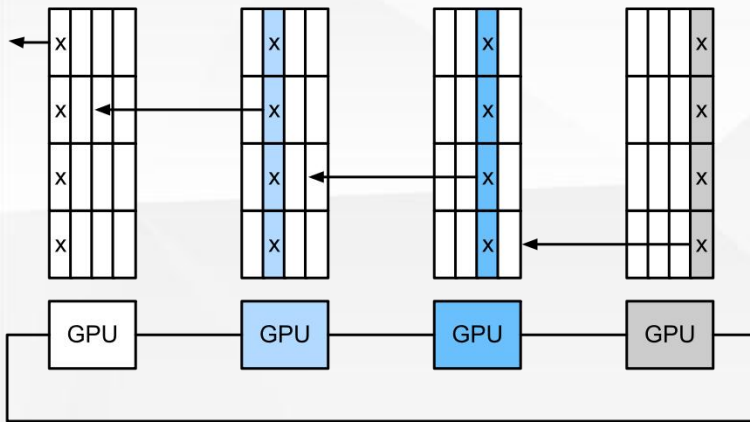


the network can be decomposed into one ring (1-2-3-4-5-6-7-8-1) with double NVLink bandwidth and into one (1-4-6-3-5-8-2-7-1) with regular bandwidth. Designing an efficient synchronization protocol in this case is nontrivial.





Ring Synchronization



given a ring of n computing nodes we can send gradients from the first to the second node. There it is added to the local gradient and sent on to the third node, and so on. After $n-1$ steps the aggregate gradient can be found in the last-visited node.

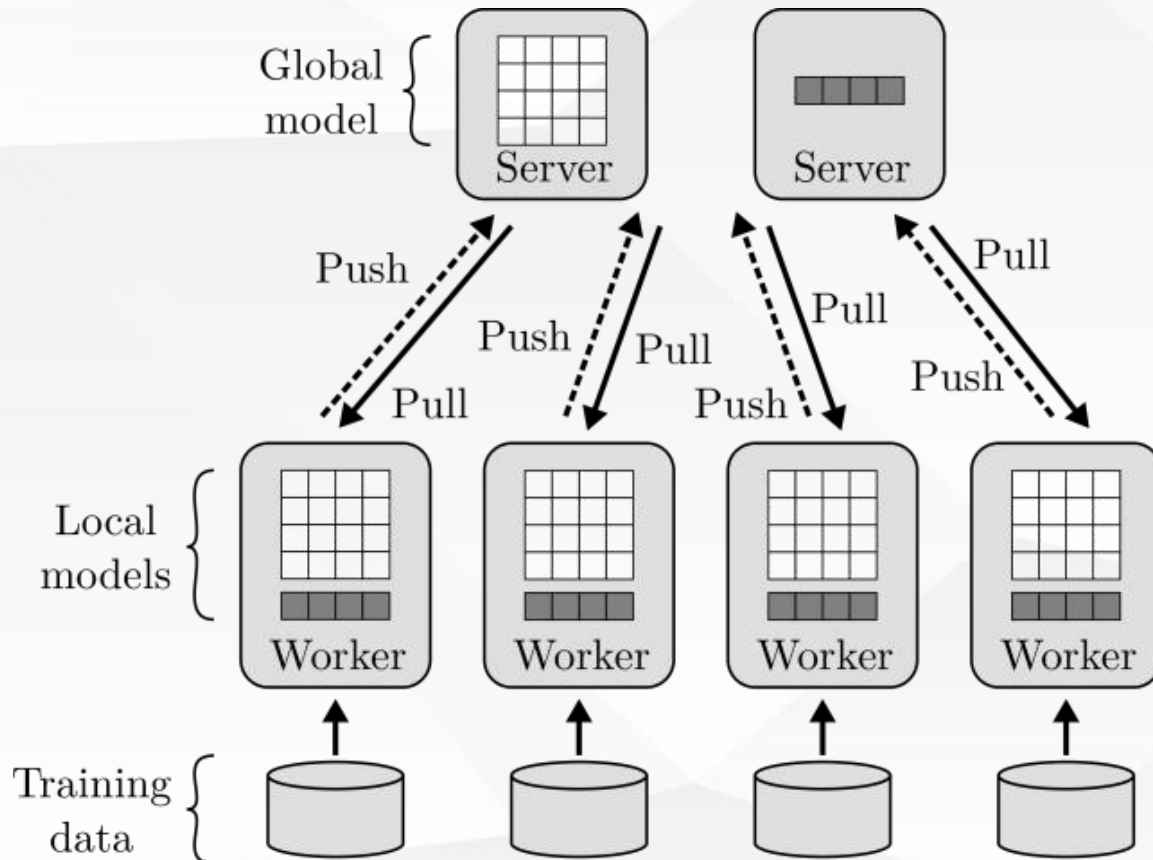
$O(n)$

broke the gradients into n chunks and started synchronizing chunk i starting at node i . Since each chunk is of size $1/n$ the total time is now $(n-1)/n \approx 1$.

$O(1)$



Distributed Methods#4



- A server (or servers) to split data, separate it and aggregate the global model.
- Many workers with separated data to train the separated models.
- Connected by high speed networks



Parameter Server

Algorithm 1 Distributed Subgradient Descent

Task Scheduler:

- 1: issue LoadData() to all workers
- 2: **for** iteration $t = 0, \dots, T$ **do**
- 3: issue WORKERITERATE(t) to all workers.
- 4: **end for**

总体的并行训练流程

1. 分发数据到每一个worker节点
2. 每一个worker节点并行执行WORKERITERATE方法，一共迭代T轮，完成模型的并行训练

Worker $r = 1, \dots, m$:

- 1: **function** LOADDATA()
- 2: load a part of training data $\{y_{i_k}, x_{i_k}\}_{k=1}^{n_r}$
- 3: pull the working set $w_r^{(0)}$ from servers
- 4: **end function**
- 5: **function** WORKERITERATE(t)
- 6: gradient $g_r^{(t)} \leftarrow \sum_{k=1}^{n_r} \partial \ell(x_{i_k}, y_{i_k}, w_r^{(t)})$
- 7: push $g_r^{(t)}$ to servers
- 8: pull $w_r^{(t+1)}$ from servers
- 9: **end function**

worker的初始化过程

- 1、每个worker载入一部分训练数据
- 2、每个worker将全部初始模型参数 w_0 从servers节点上拉下来

worker节点的迭代计算过程

- 1、仅利用本节点的训练数据计算梯度
- 2、将计算好的梯度 $g_r(t)$ push到servers节点
- 3、将新一轮的模型参数 $w(t+1)$ 拉到本地worker节点

Servers:

- 1: **function** SERVERITERATE(t)
- 2: aggregate $g^{(t)} \leftarrow \sum_{r=1}^m g_r^{(t)}$
- 3: $w^{(t+1)} \leftarrow w^{(t)} - \eta (g^{(t)} + \partial \Omega(w^{(t)}))$
- 4: **end function**

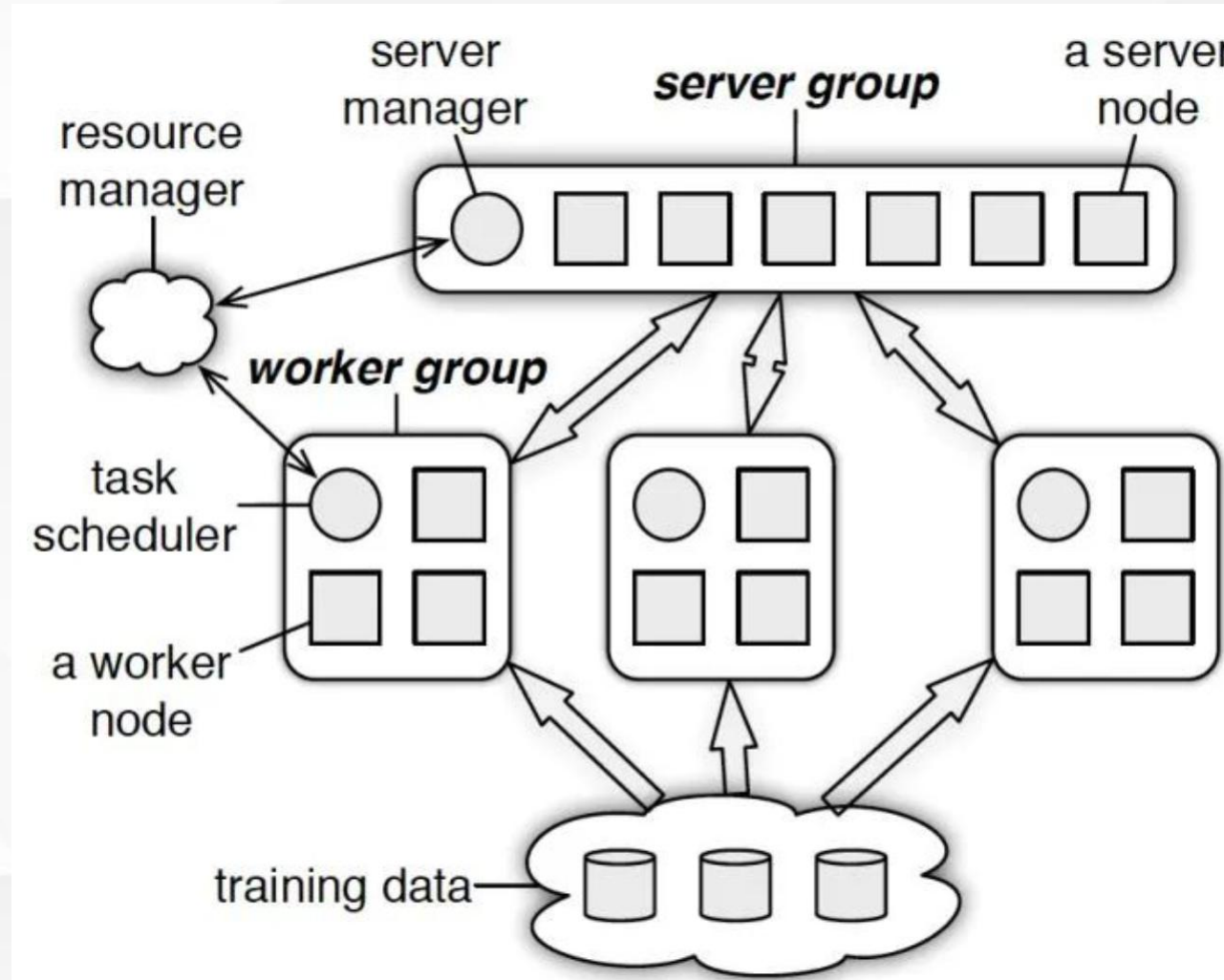
server节点的迭代计算过程

- 1、在收到m个worker节点计算的梯度后，汇总梯度形成总梯度
- 2、利用汇总梯度 $g(t)$ ，融合正则化项梯度，计算出新梯度 $w(t+1)$



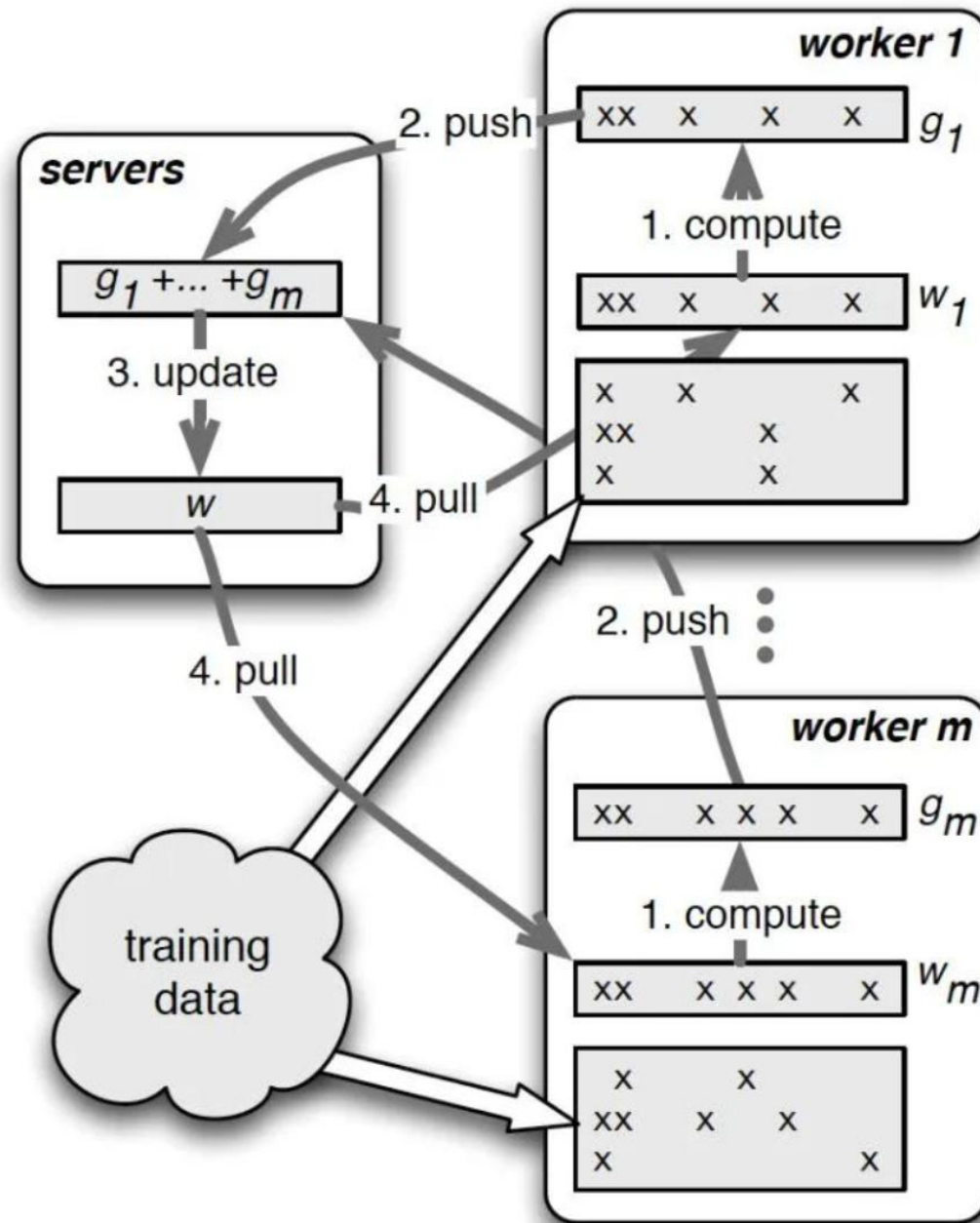


Parameter Server Physical architecture





Parameter Server Parallel Training

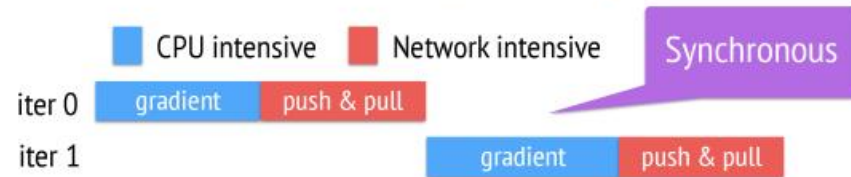




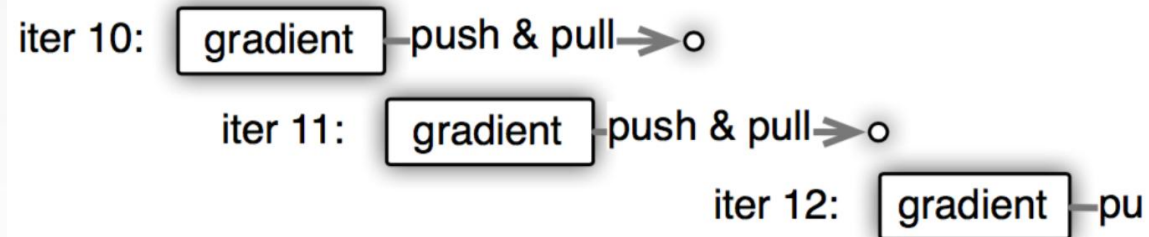
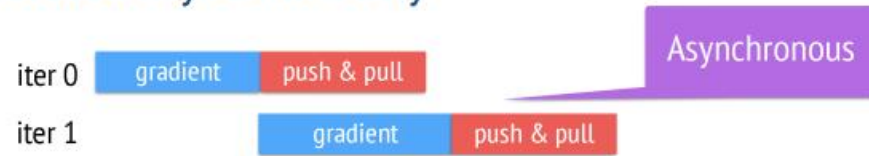
Asynchronous tasks and dependency



“execute-after-finished” dependency



executed asynchronously



Bulk Synchronous Execution

Machine



Machine



Machine

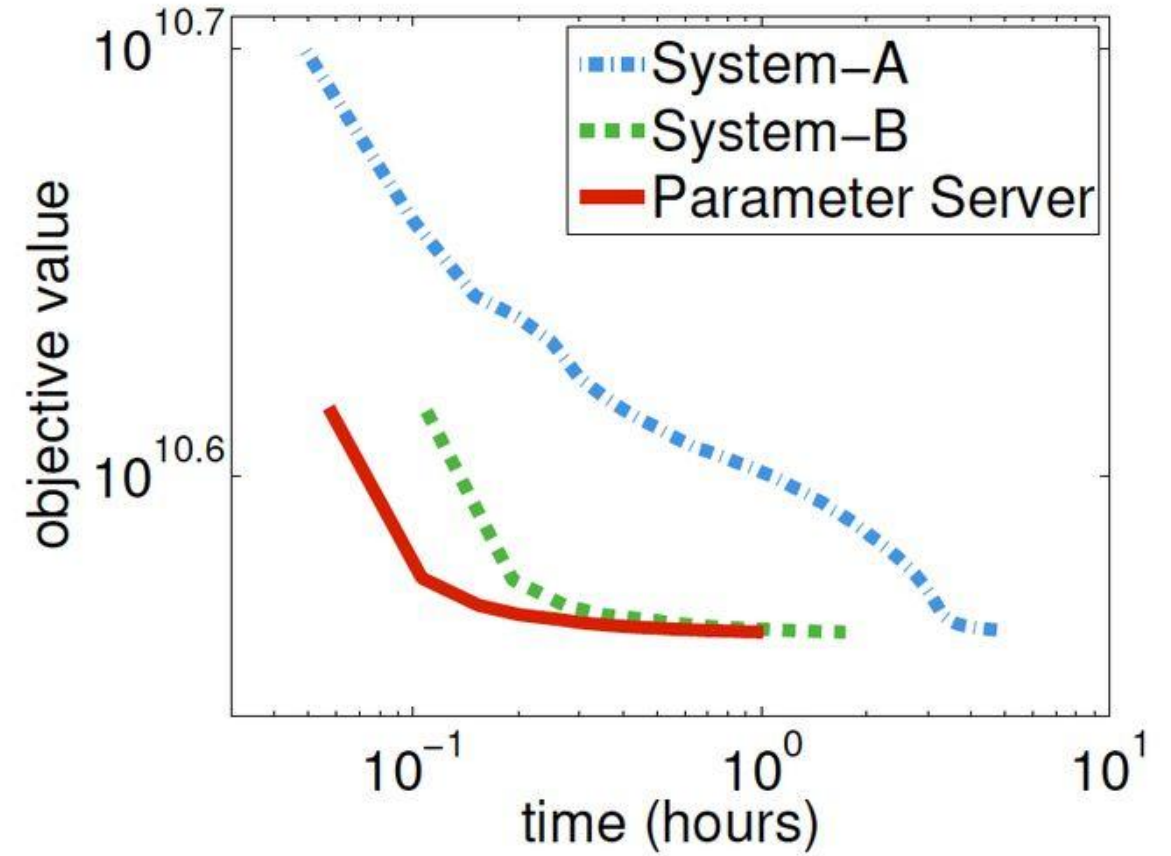
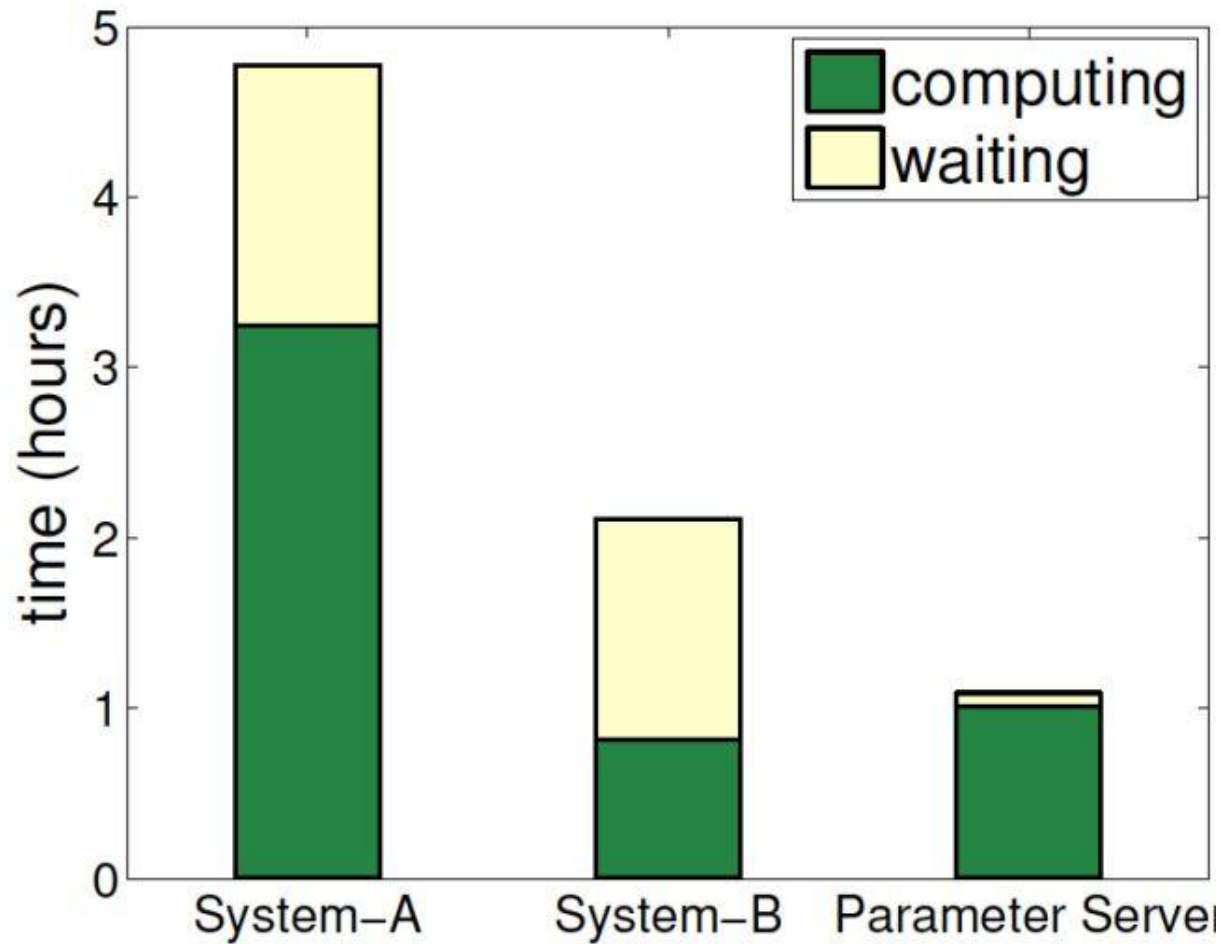


Any push or pull request can be a task, so can be a remote function that is executing. Tasks are generally asynchronous in nature and programs/applications can continue executing after issuing the task.



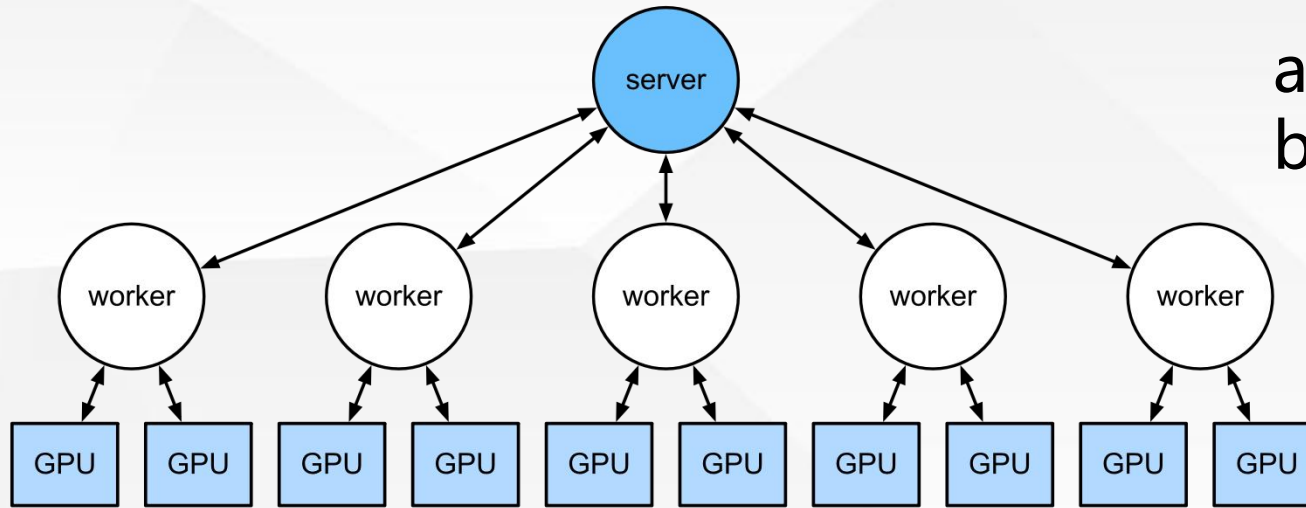


PS vs Bulk Synchronous system



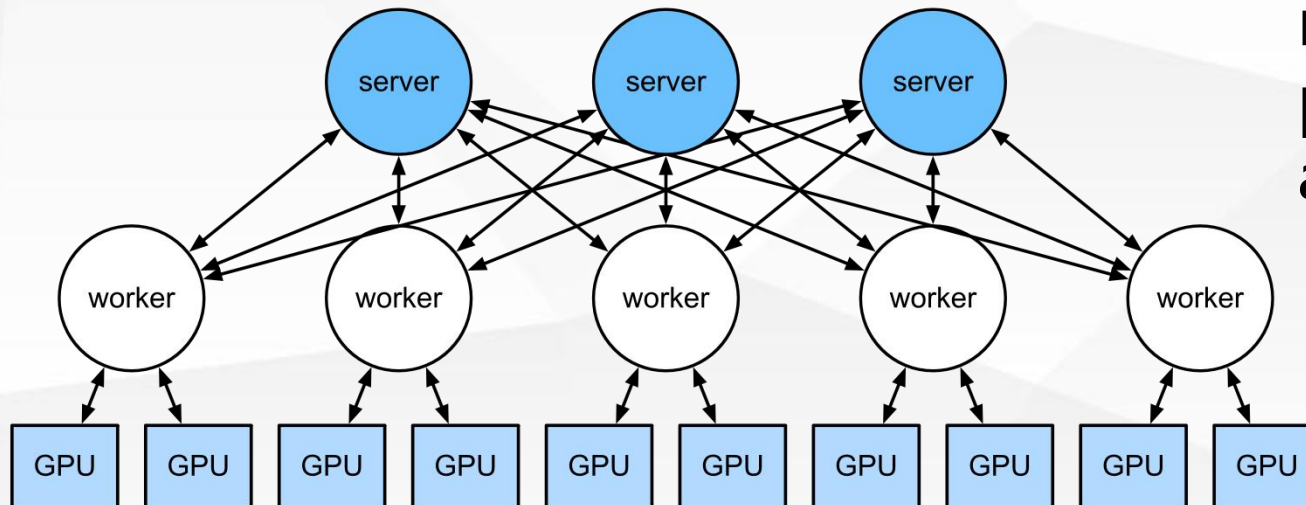


Coordinate of Multi-PS



single parameter server

a single parameter server is a bottleneck since its bandwidth is finite.

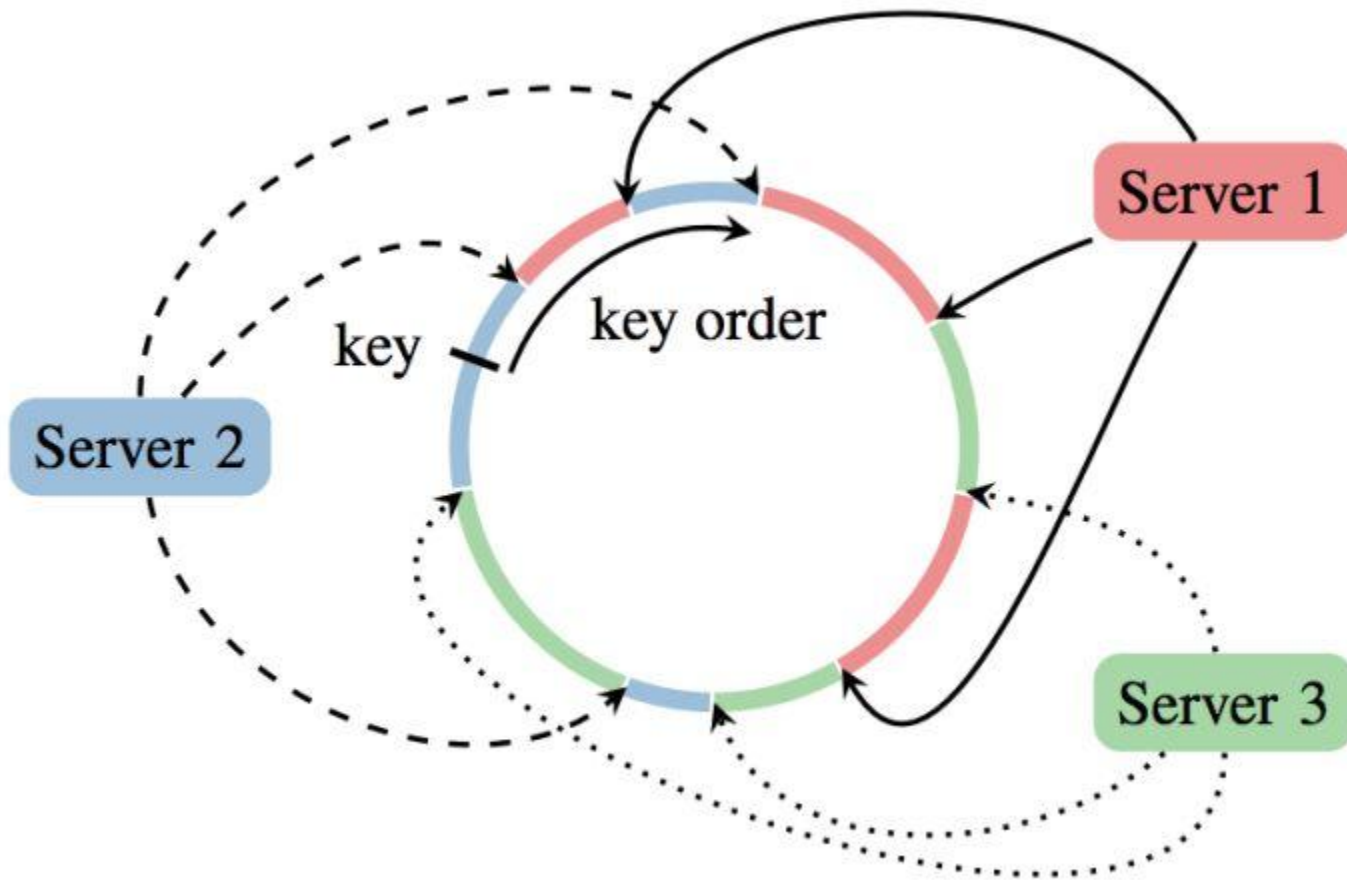


multiple servers

multiple parameter servers store parts of the parameters with aggregate bandwidth.



Consistent Hash



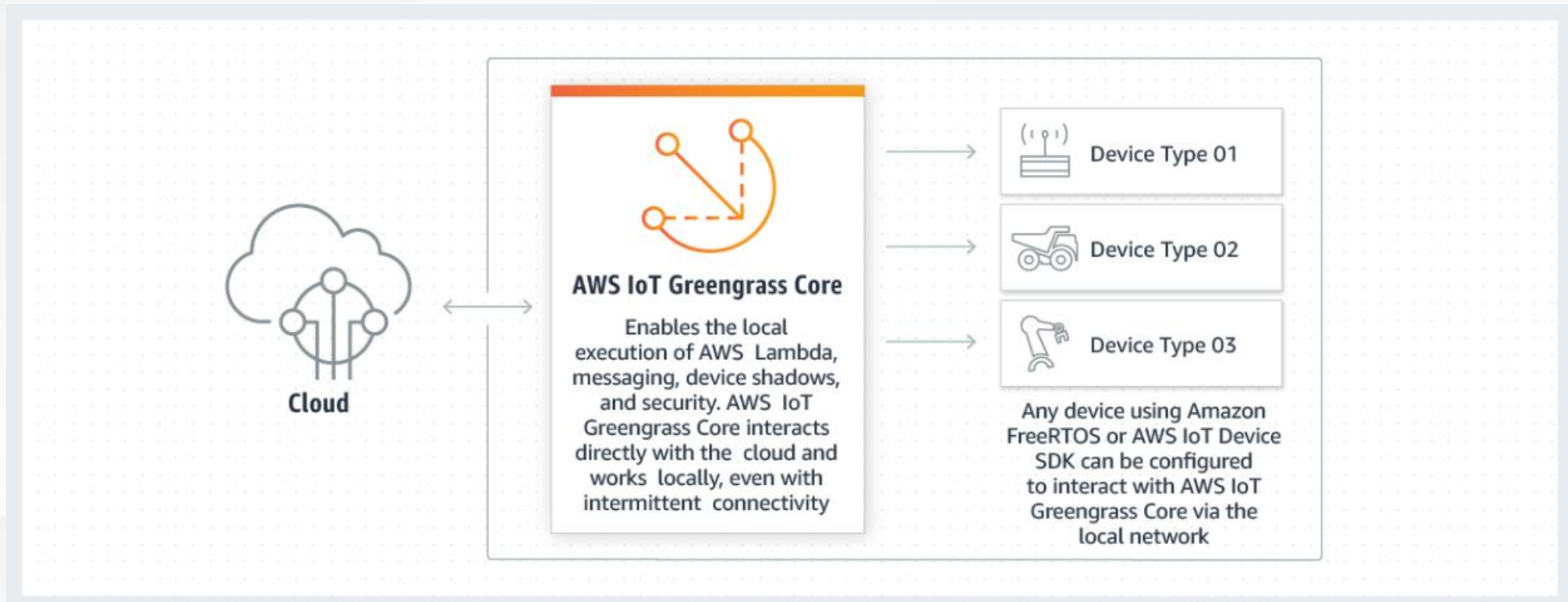
A message

(time signature, len(range), k, v)



Hybrid Cloud-Edge Computing

- Combine cloud servers with edge devices for hierarchical model training.
- Example: AWS IoT Greengrass integrates edge nodes to reduce communication latency.





Serverless Distributed Learning

- Leverage serverless frameworks (e.g., AWS Lambda, Google Cloud Functions) for dynamic resource allocation.
- Advantages: Pay-per-use, auto-scaling, and reduced infrastructure overhead.



02

Privacy and Security Issues in Deep Learning

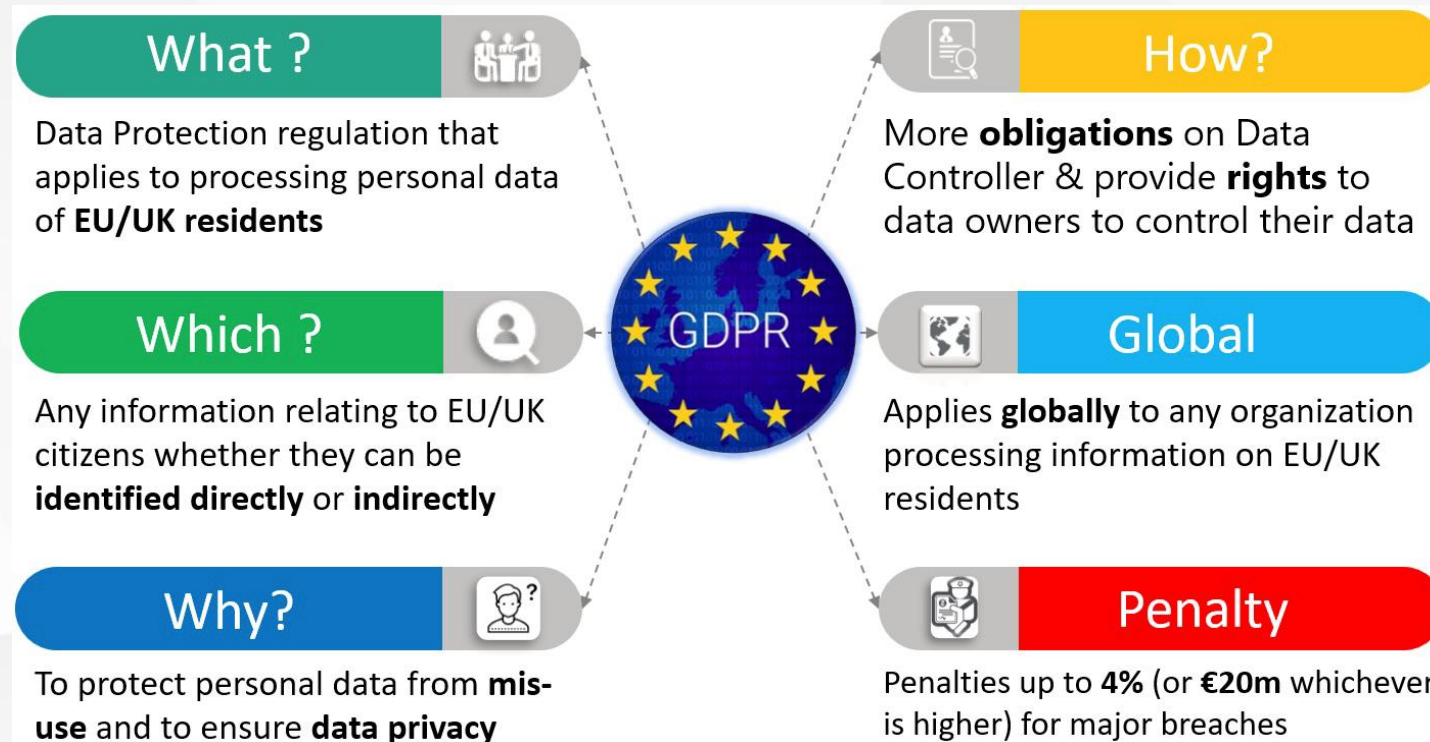


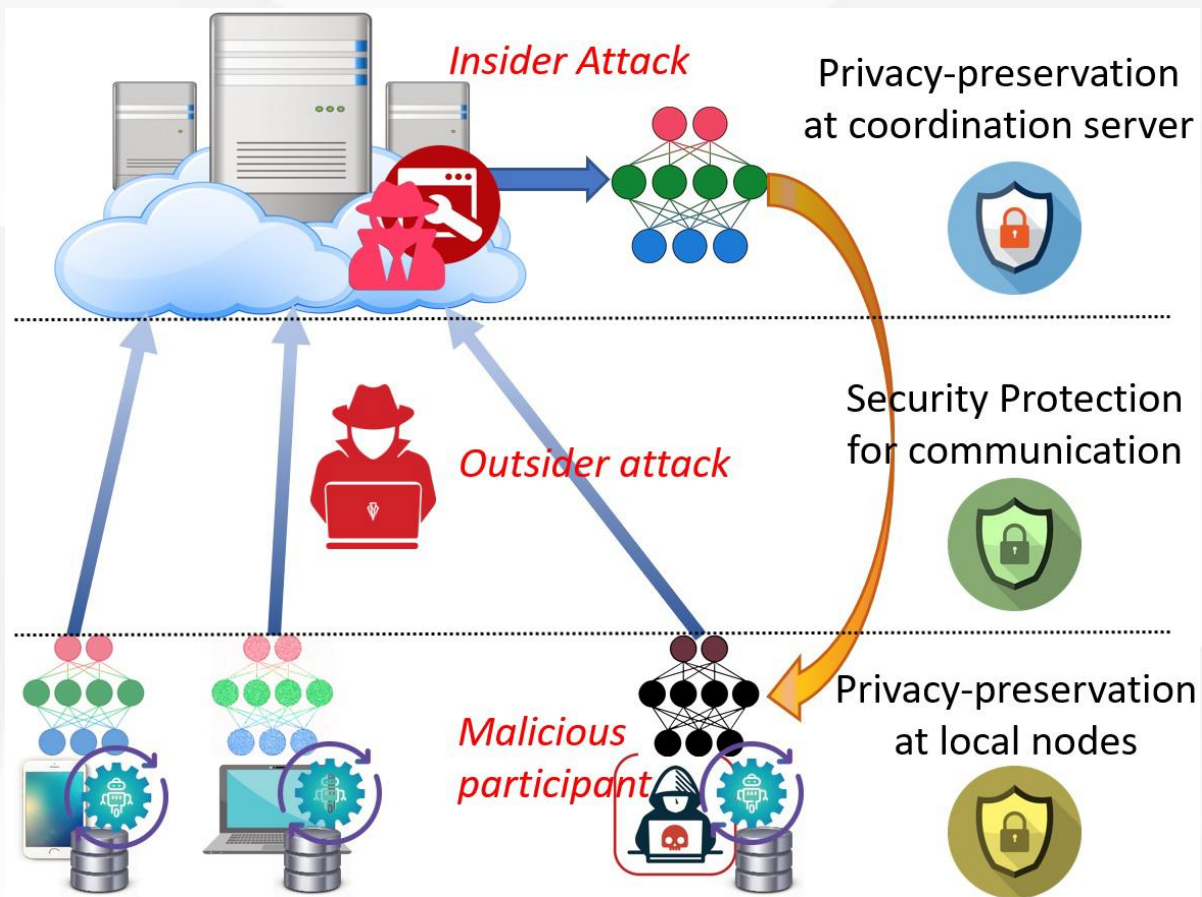


Privacy Principles for Learning and Analytics



- ④ The challenges are not only due to transporting high-volume, high-velocity, high-veracity, and heterogeneous data across organisations but also the data protection regulations and restrictions such as the EU General Data Protection Regulation (GDPR)





- ① (i) prevent insiders at the server from conducting inference attacks
- ① (ii) prevent Byzantine participants from conducting model poisoning.



Threats and Attacks

④ Privacy inference attacks

Existing work suggested that adversaries can infer different levels of sensitive information from the updated gradients

④ Eavesdropping attacks

The adversaries located in the communication channel between central server and local workers can launch eavesdropping attacks. The adversaries can steal or tamper some meaningful information, such as model weights or gradients, in each communication.

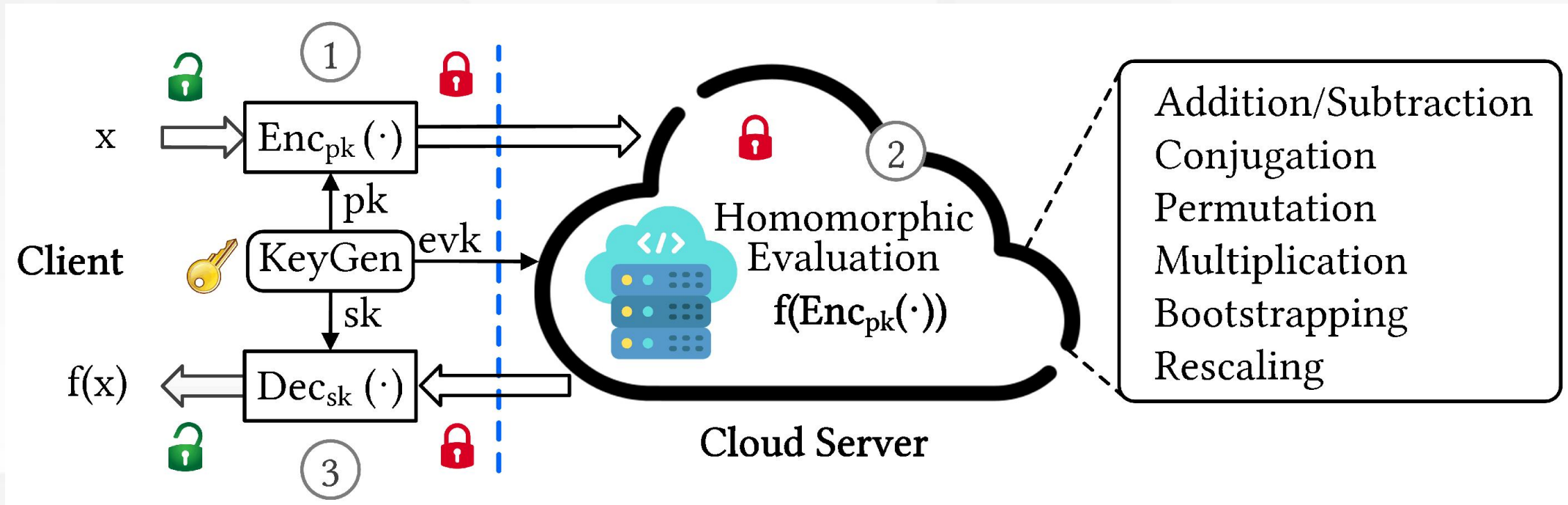
④ Poisoning attacks

Poisoning attacks on machine learning models have been widely studied. These attacks occur in the training phase against FL. On the one hand, adversaries can impair the performance of the final global model on untargeted tasks. On the other hand, adversaries can inject a backdoor into the final global model.



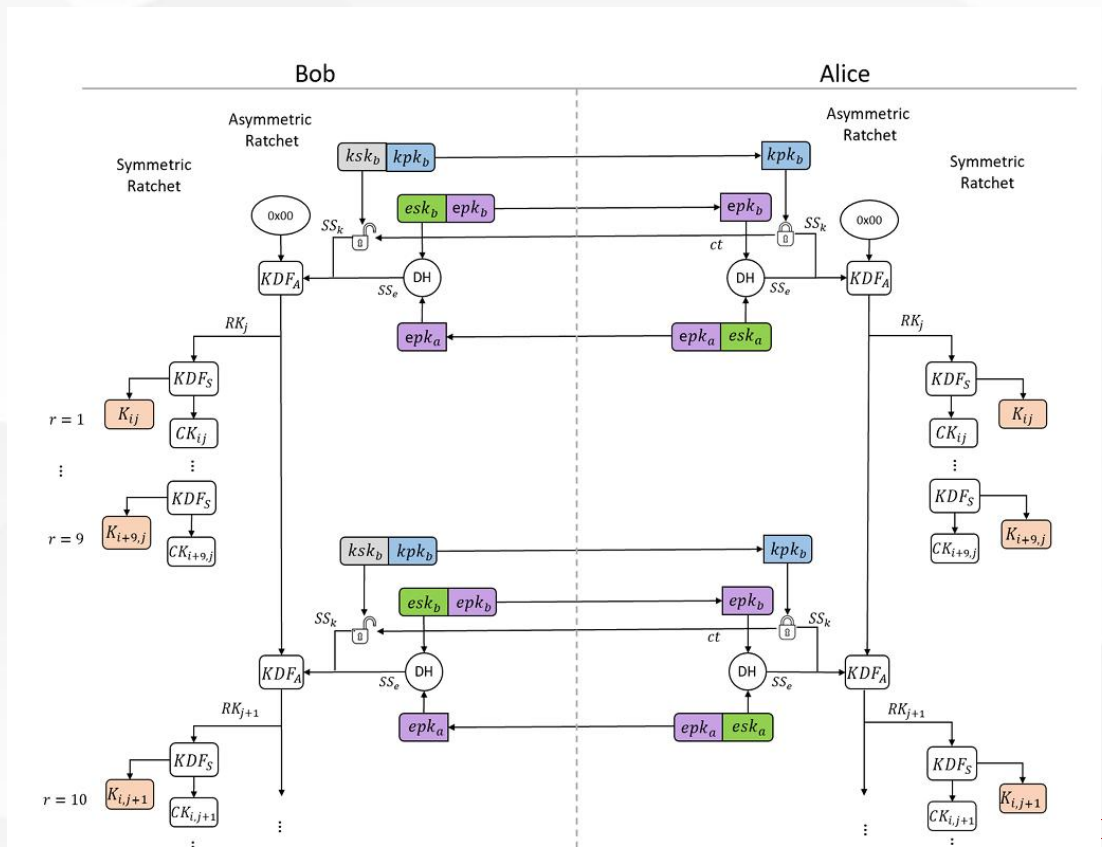
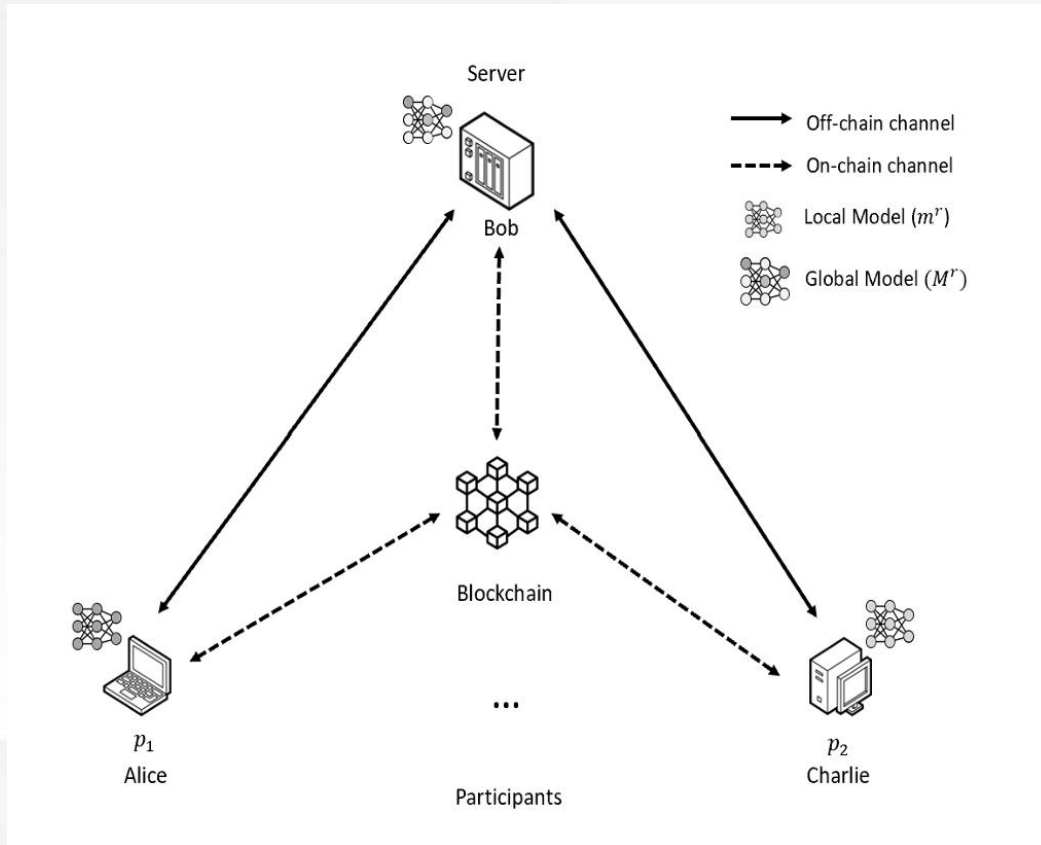
④ Fully Homomorphic Encryption (FHE)

- Recent breakthroughs in FHE efficiency, e.g., Microsoft's SEAL 4.0 supports 10x faster encryption for neural network gradients.



Post-Quantum Cryptography for Distributed Learning

- Protect Distributed Learning against quantum computing attacks using lattice-based cryptography (e.g., NIST PQC finalists).



03

Strength of Federated Learning





What is Federated Learning?

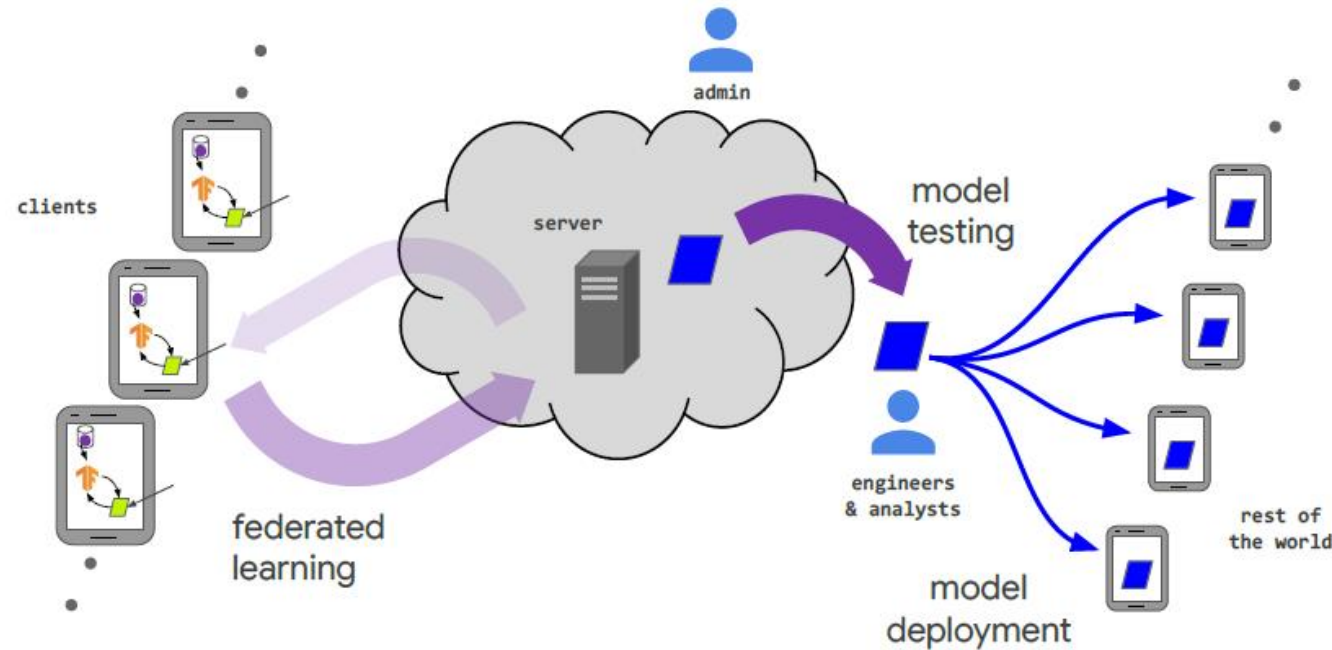
The term federated learning was introduced in 2016 by McMahan et al:

“We term our approach Federated Learning, since the learning task is solved by a loose federation of participating devices (which we refer to as clients) which are coordinated by a central server.”

A broader definition

Federated learning is a machine learning setting where multiple entities (clients) collaborate in solving a machine learning problem, under the coordination of a central server or service provider. Each client’s raw data is stored locally and not exchanged or transferred; instead, focused updates intended for immediate aggregation are used to achieve the learning objective





1. Problem identification: The model engineer identifies a problem to be solved with FL.

2. Client instrumentation: If needed, the clients (e.g. an app running on mobile phones) are instrumented to store locally (with limits on time and quantity) the necessary training data. In many cases, the app already will have stored this data (e.g. a text messaging app must store text messages, a photo management app already stores photos). However, in some cases additional data or metadata might need to be maintained, e.g. user interaction data to provide labels for a supervised learning task.

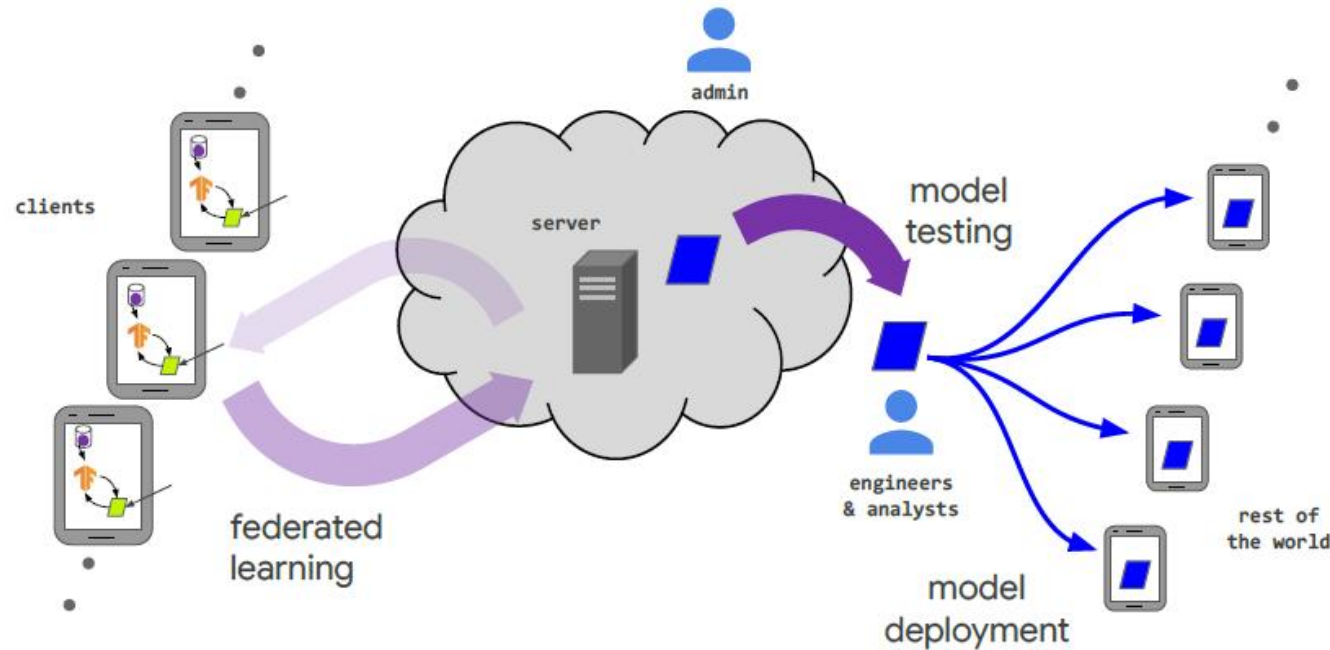


Life Cycle of Federated Learning

3. Simulation prototyping (optional):

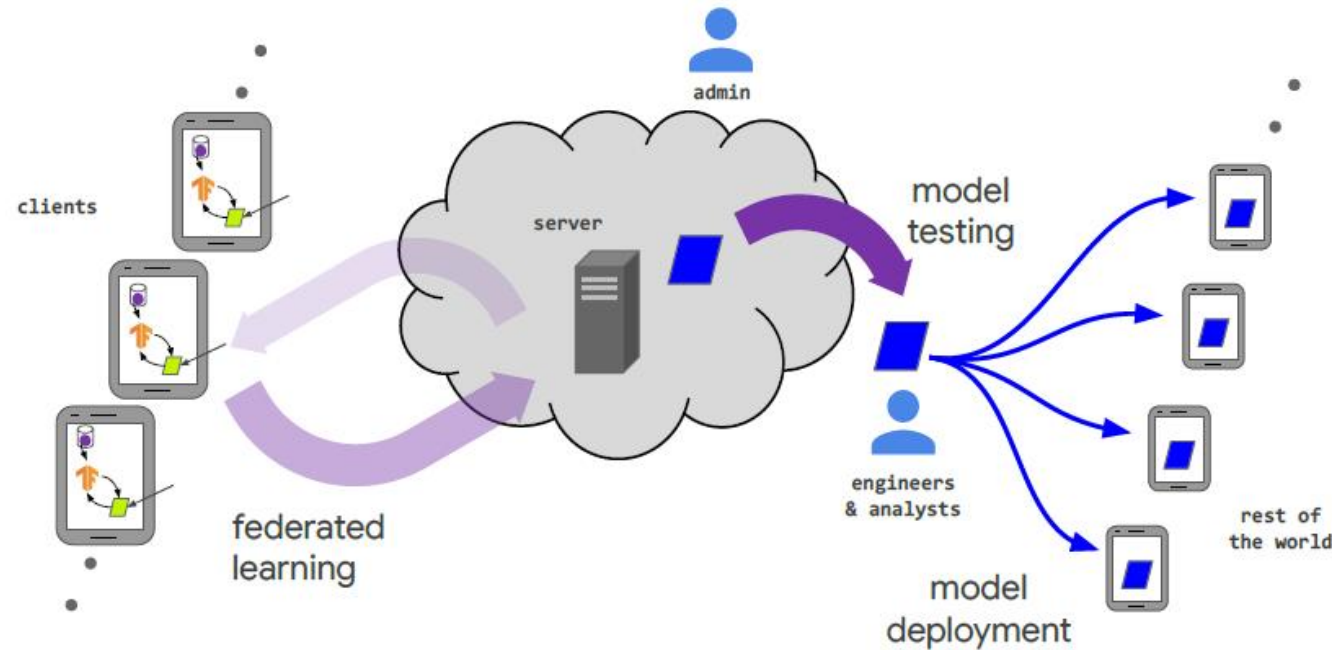
The model engineer may prototype model architectures and test learning hyperparameters in an FL simulation using a proxy dataset.

4. **Federated model training:** Multiple federated training tasks are started to train different variations of the model, or use different optimization hyperparameters.





Life Cycle of Federated Learning

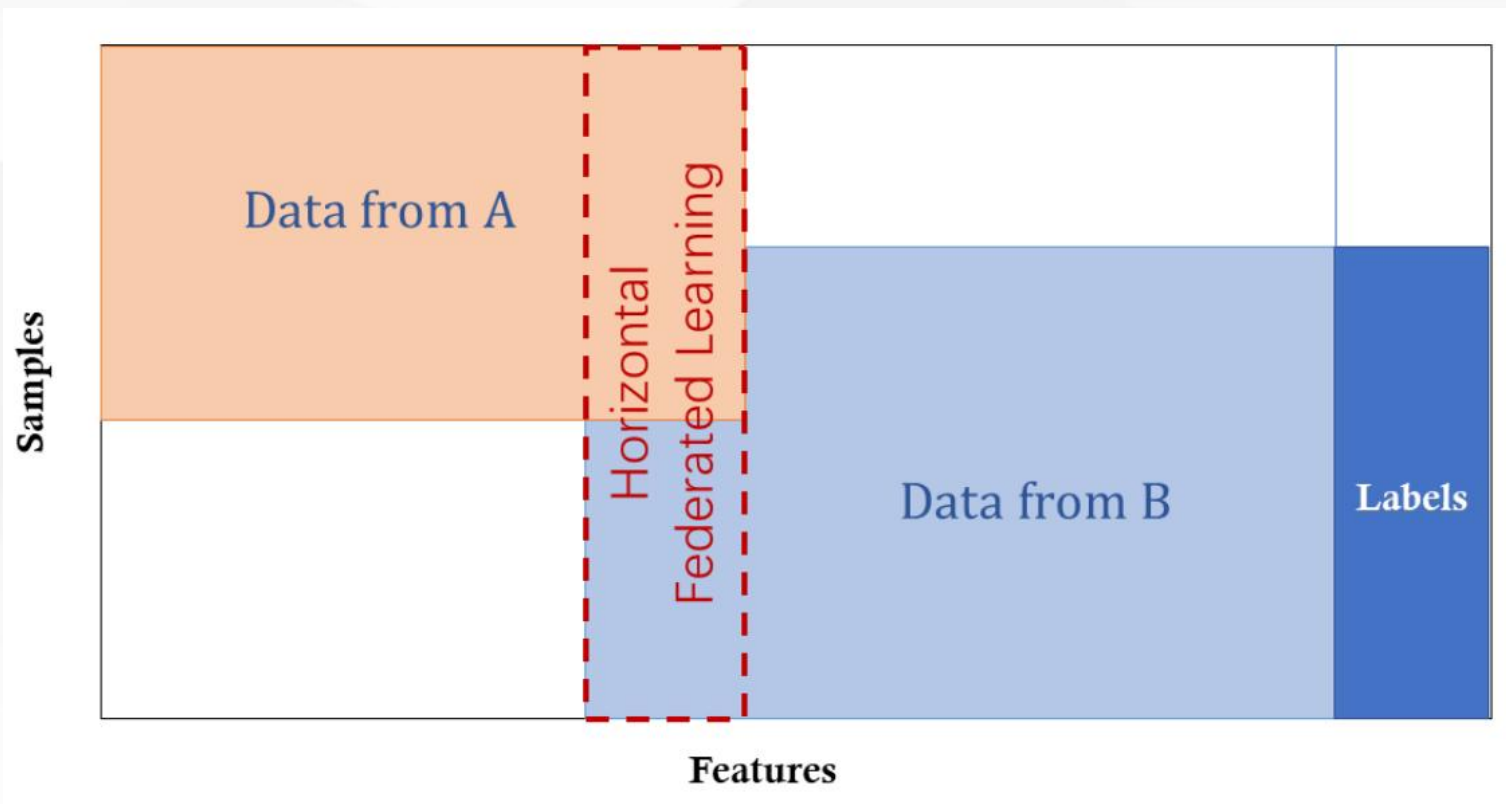


5. **(Federated) model evaluation**: After the tasks have trained sufficiently (typically a few days, see below), the models are analyzed and good candidates selected. Analysis may include metrics computed on standard datasets in the datacenter, or federated evaluation wherein the models are pushed to held-out clients for evaluation on local client data.

6. **Deployment**: Finally, once a good model is selected, it goes through a standard model launch process, including manual quality assurance, live A/B testing



Classification

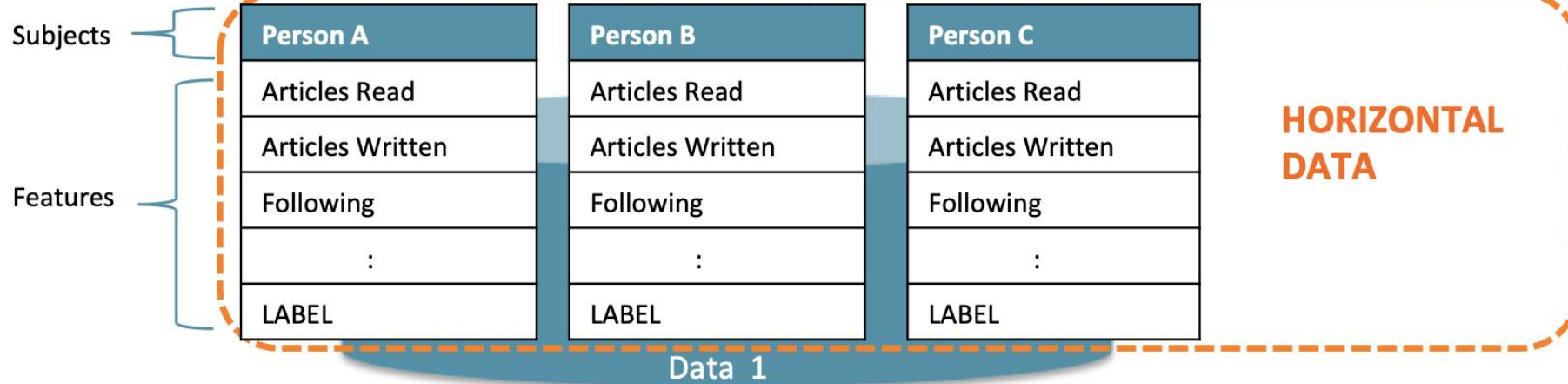


“Horizontal federated learning, or sample-based federated learning, is introduced in the scenarios that data sets share the same feature space but different in sample.”



Online publisher of
blogs, curated content.

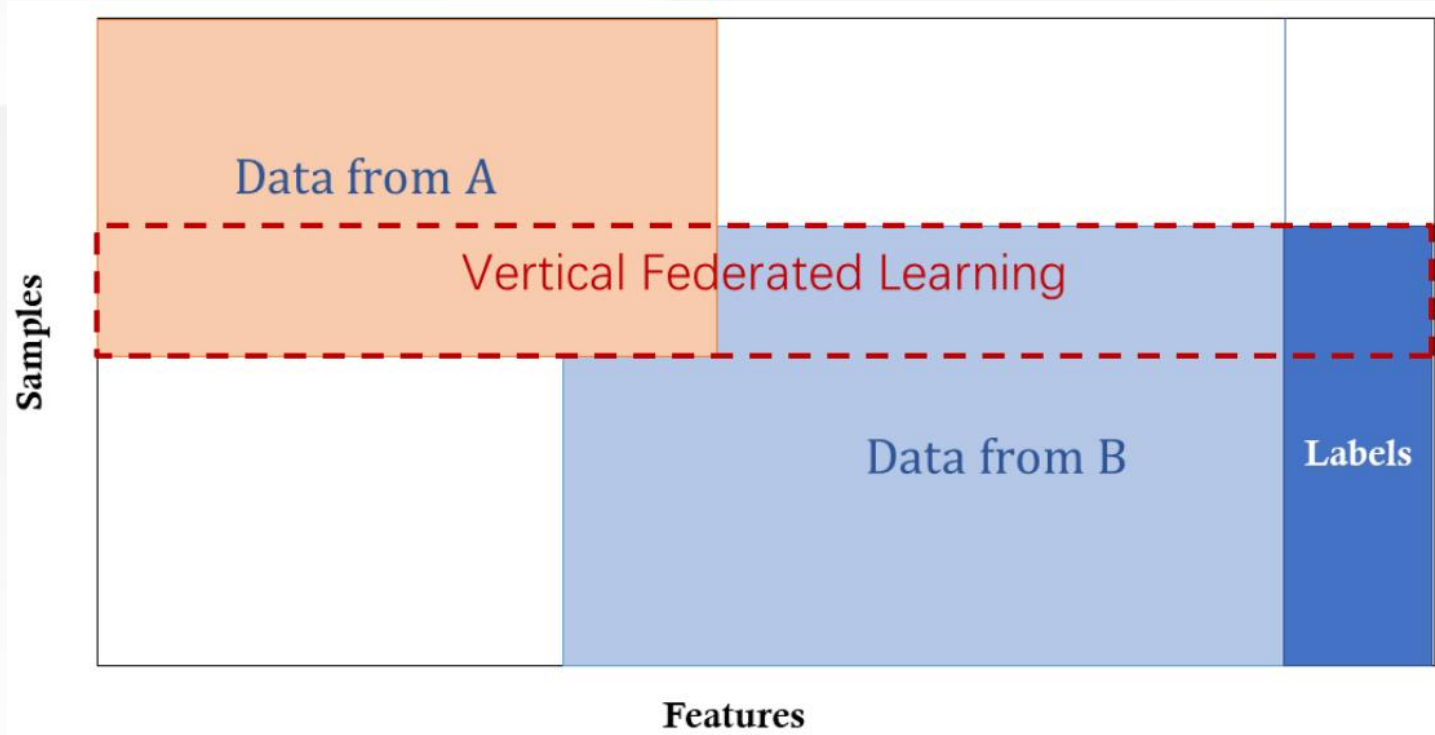
Large



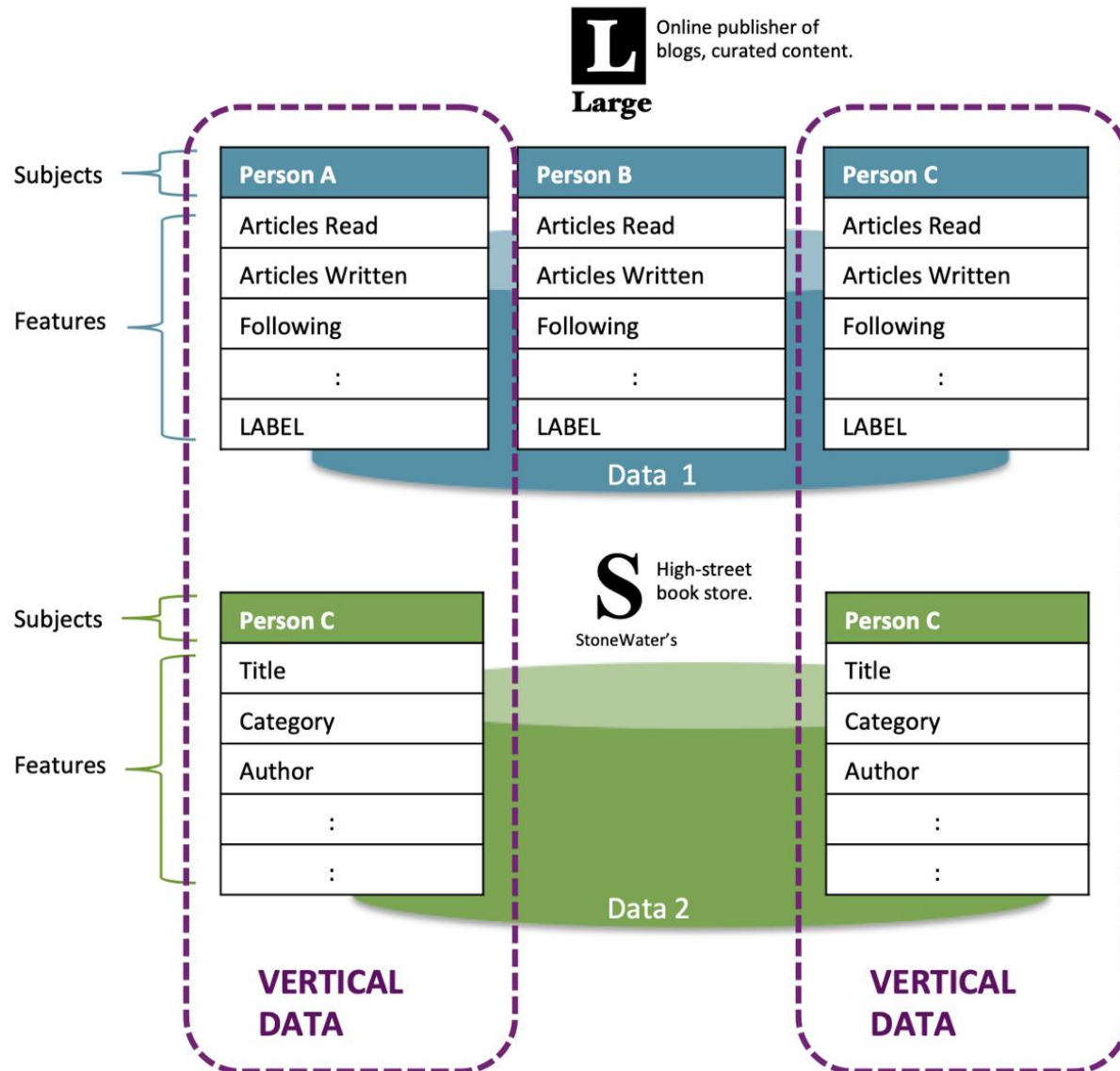
- With horizontal data, rows of data are available with a consistent set of features. This is exactly the type of data you'd feed into a supervised machine learning task.



Classification



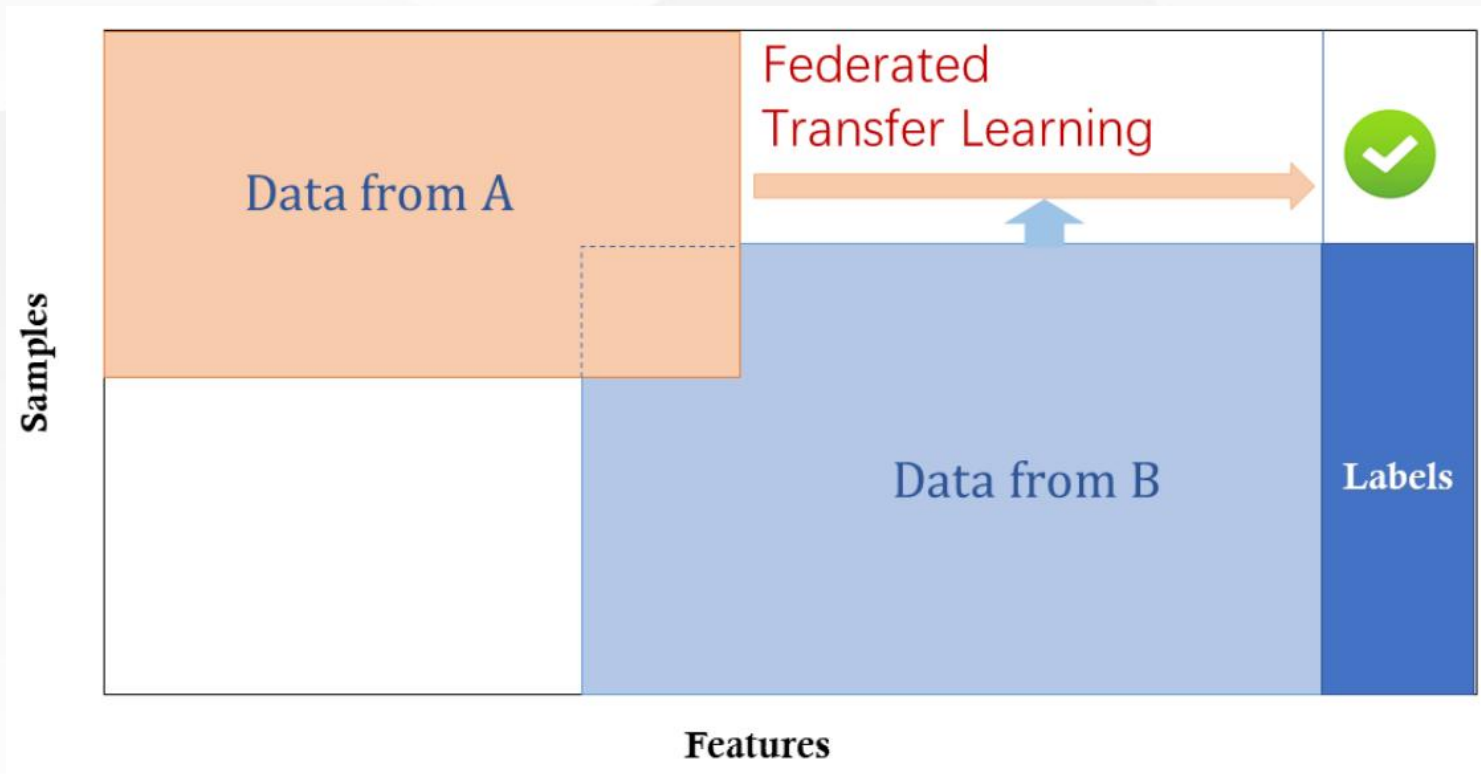
- “Vertical federated learning or feature-based federated learning ... is applicable to the cases that two data sets share the same sample ID space but differ in feature space.”



 fictitious high street book retailer *StoneWater's* have some of the same customers as online blogging curator *Large* (*also totally fictitious*) and capture different features such as book 'Title', 'Category', 'Author' from each purchase.

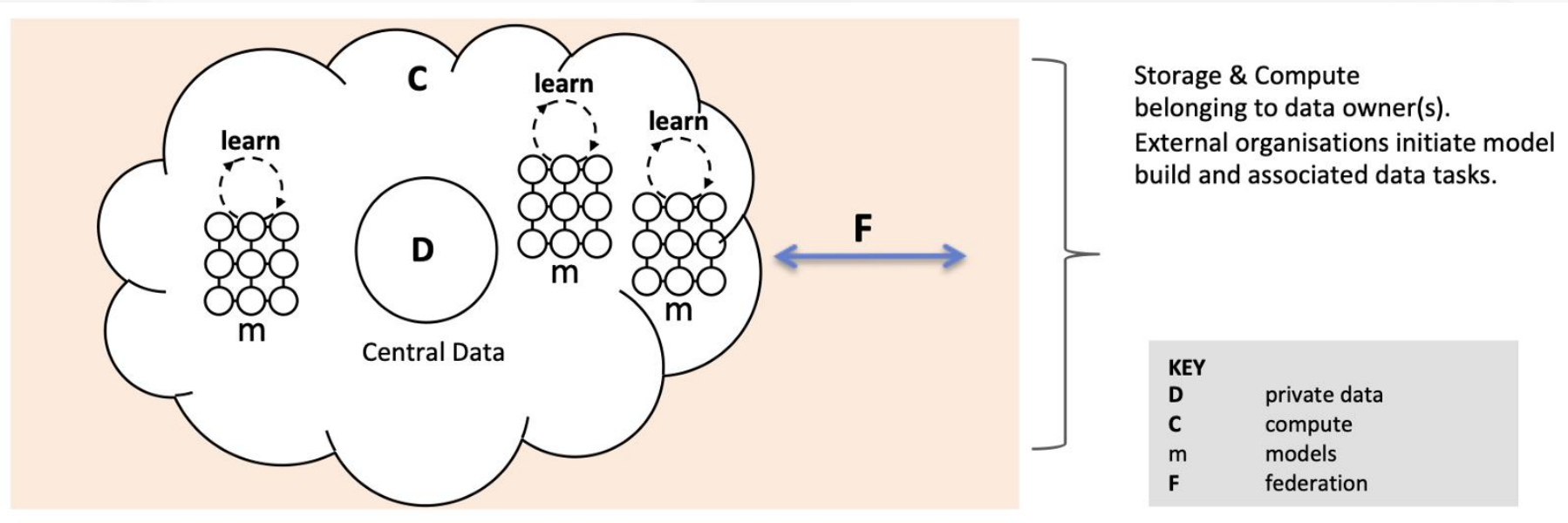


Classification



- “Transfer federated learning is applicable to the cases that two data sets share some same sample ID space and feature space.”

Data Centric Federated Learning



- ⊗ This is a newer, emerging type of Federated Learning, and in some ways may be outgrowing the Federated term, having a more peer-to-peer feel.
- ⊗ An owner, or in future—owners, of private data can provide access for external organisations to build models on their data without sharing that data.



Comparison

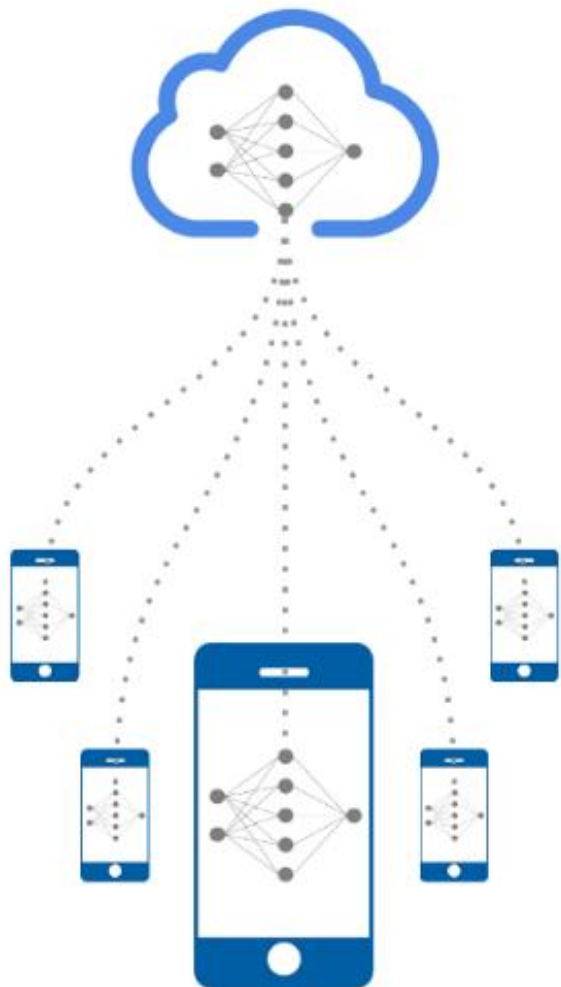
TABLE 1: TYPICAL FL SETTINGS AND OF TRADITIONAL DISTRIBUTED LEARNING

	DATACENTER DISTRIBUTED LEARNING	CROSS-SILO FEDERATED LEARNING	CROSS-DEVICE FEDERATED LEARNING
Setting	Training a model on a large but “flat” dataset. Clients are compute nodes in a single cluster or datacenter.	Training a model on siloed data. Clients are different organizations (e.g., medical or financial) or datacenters in different geographical regions.	The clients are a very large number of mobile or IoT devices.
Data distribution	Data is centrally stored, so it can be shuffled and balanced across clients. Any client can read any part of the dataset.	Data is generated locally and remains decentralized. Each client stores its own data and cannot read the data of other clients. Data is not independently or identically distributed.	
Orchestration	Centrally orchestrated.	A central orchestration server/service organizes the training, but never sees raw data.	
Distribution scale	Typically 1 - 1000 clients.	Typically 2 - 100 clients.	Up to 10^{10} clients.
Client properties	Clients are reliable and almost always available to participate in computations. Clients may be directly addressed, and can maintain state across computation rounds.		Clients are often unavailable and can only be accessed by random sampling from available devices. For large populations a single client will typically only participate once in a given computation.





Strengths



Hyper-Personalized



Low Cloud Infra Overheads



Minimum Latencies



Privacy Preserving



Strengths



- FL enables devices like mobile phones to collaboratively learn a shared prediction model while keeping the training data on the device instead of requiring the data to be uploaded and stored on a central server.
- Moves model training to the edge, namely devices such as smartphones, tablets, IoT, or even “organizations” like hospitals that are required to operate under strict privacy constraints. Having personal data remain local is a strong security benefit.
- Makes real-time prediction possible, since prediction happens on the device itself. FL reduces the time lag that occurs due to transmitting raw data back to a central server and then shipping the results back to the device.
- Since the models reside on the device, the prediction process works even when there is no internet connectivity.
- FL reduces the amount of hardware infrastructure required. FL uses minimal hardware and what is available in mobile devices is more than enough to run the FL models.





Various technologies for Privacy

Technologies	Characteristic
Differential Privacy (local, central, shuffled, aggregated, and hybrid models)	A quantification of how much information could be learned about an individual from the output of an analysis on a dataset that includes the user. Algorithms with differential privacy necessarily incorporate some amount of randomness or noise, which can be tuned to mask the influence of the user on the output.
Secure Multi-Party Computation	Two or more participants collaborate to simulate, through cryptography, a fully trusted third party who can: • Compute a function of inputs provided by all the participants; • Reveal the computed value to a chosen subset of the participants, with no party learning anything further.



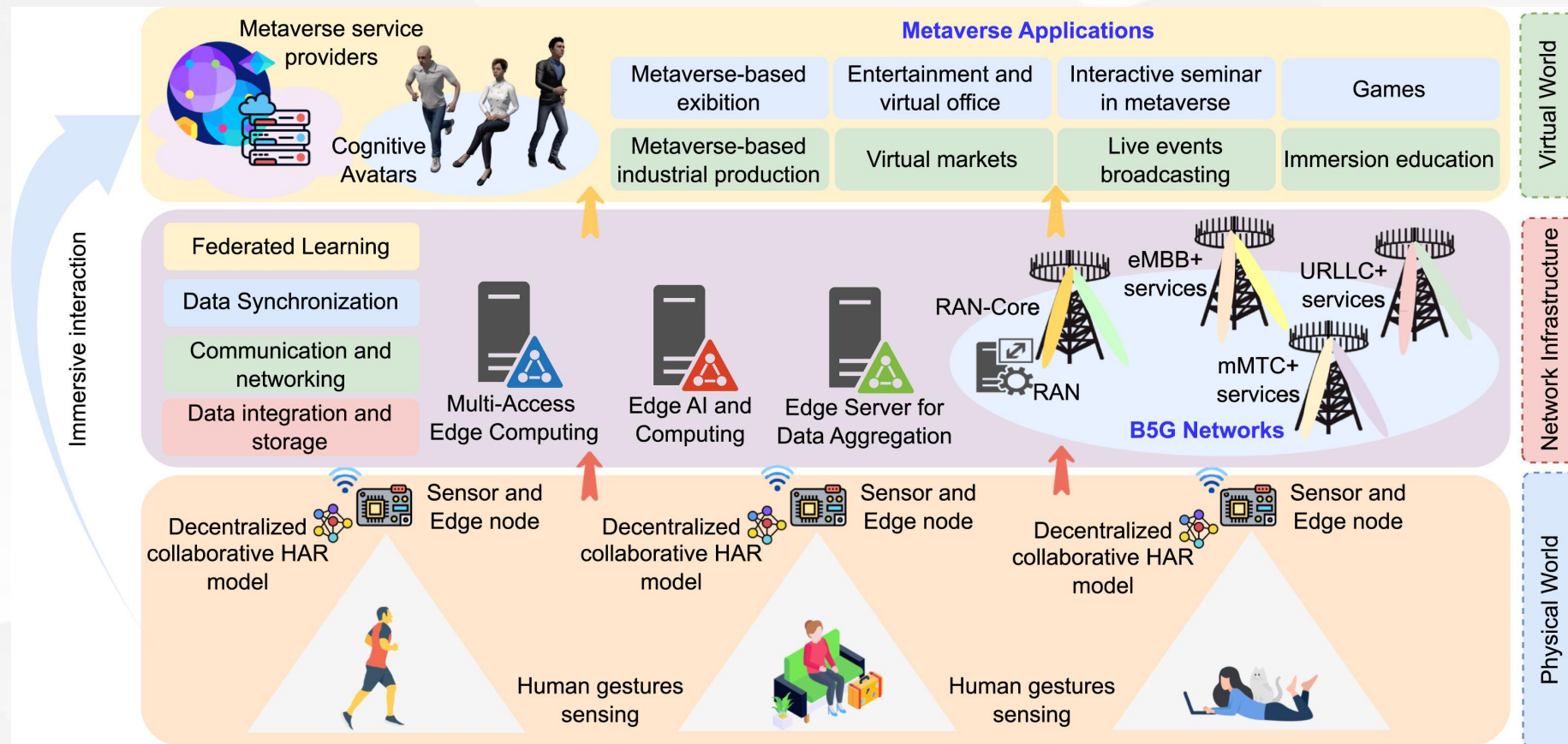
Various technologies for Privacy

Technologies	Characteristic
Homomorphic Encryption	Enables a party to compute functions of data to which they do not have plain-text access, by allowing mathematical operations to be performed on ciphertexts without decrypting them. Arbitrarily complicated functions of the data can be computed this way ("Fully Homomorphic Encryption") though at greater computational cost.
Trusted Execution Environments (secure enclaves)	• Confidentiality: The state of the code's execution remains secret, unless the code explicitly publishes a message; • Integrity: The code's execution cannot be affected, except by the code explicitly receiving an input; • Measurement/Attestation: The TEE can prove to a remote party what code (binary) is executing and what its starting state was, defining the initial conditions for confidentiality and integrity.

Advanced Federated Learning

Interactive Metaverse Avatars

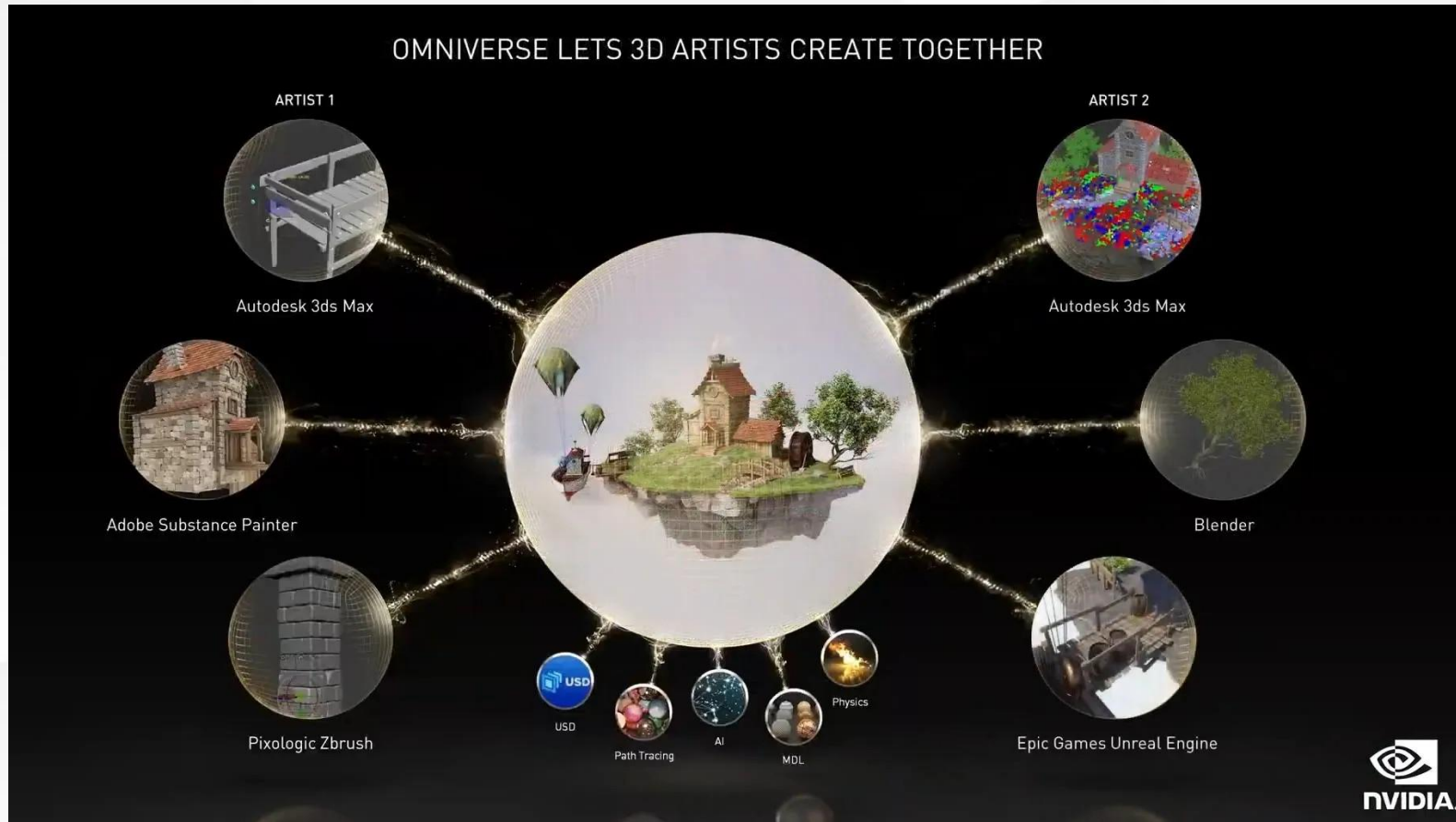
- Federated learning (FL) enables edge-based human activity recognition for interactive metaverse avatars with high accuracy.





Virtual Environment Simulation

- Collaborative training of physics engines using FL to ensure consistent interactions in virtual worlds.



- ① 数据最高管理部门——国家数据局成立
- ② 负责协调推进数据基础制度建设，统筹数据资源整合共享和开发利用，统筹推进数字中国、数字经济、数字社会规划和建设等，由国家发展和改革委员会管理。
- ③ 国家数据局的成立或将促进以下行业的发展：
 - 数据收集（各行业数字化转型）
 - 数据储存(国资云，底层算力基础设施)
 - 数据处理（数据安全与隐私计算）
 - 数据定价（数据资产化）
 - 数据流转(数据交易机构、公共数据交易)



新华社

23-3-7 16:46 来自 微博网页版 已编辑

【#组建国家数据局#】根据国务院关于提请审议国务院机构改革方案的议案，组建国家数据局。负责协调推进数据基础制度建设，统筹数据资源整合共享和开发利用，统筹推进数字中国、数字经济、数字社会规划和建设等，由国家发展和改革委员会管理。

将中央网络安全和信息化委员会办公室承担的研究拟订数字中国建设方案、协调推动公共服务和社会治理信息化、协调促进智慧城市建设、协调国家重要信息资源开发利用与共享、推动信息资源跨行业跨部门互联互通等职责，国家发展和改革委员会承担的统筹推进数字经济发展、组织实施国家大数据战略、推进数据要素基础制度建设、推进数字基础设施布局建设等职责划入国家数据局。#全国两会时光#



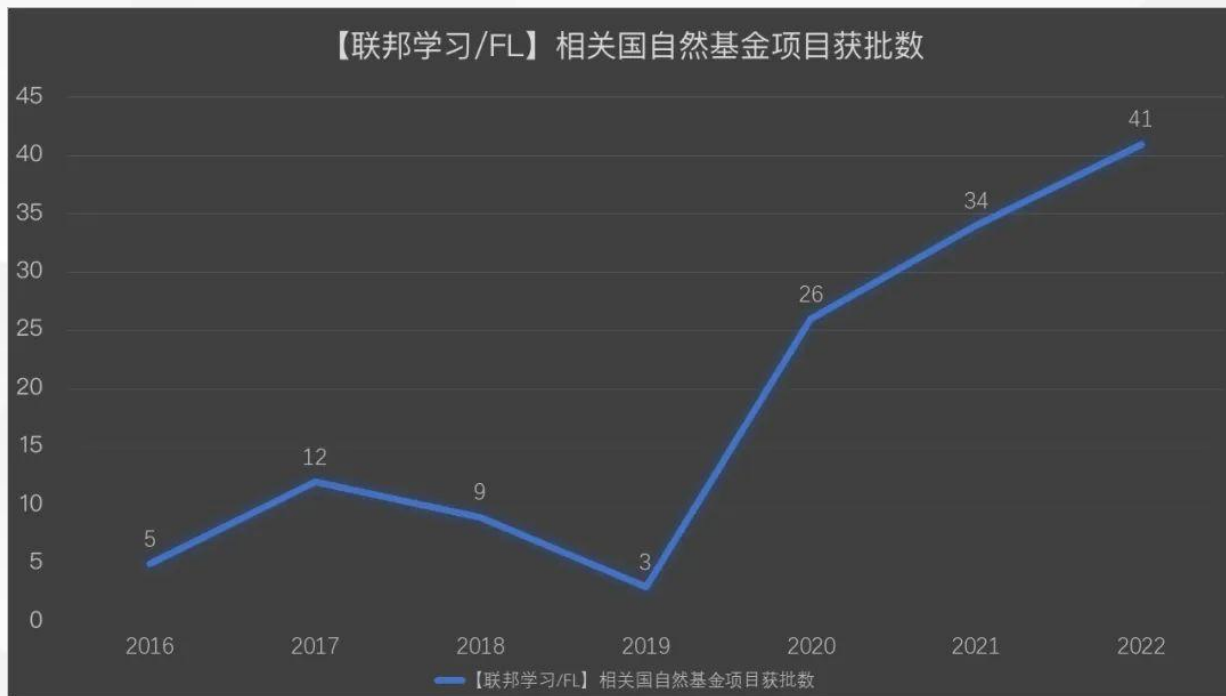


	安全多方计算	联邦学习	同态加密	可信执行环境	合成数据	差分隐私	秘密共享
波士顿女性劳动力							
欧洲统计系统							
欧盟统计局							
印度尼西亚旅游部							
意大利国家统计研究所							
英国国家统计局							
三星SDS (韩国)							
加拿大统计局							
韩国统计局							
荷兰统计局							
联合国欧洲经济委员会							
美国人口普查局							
美国教育部							





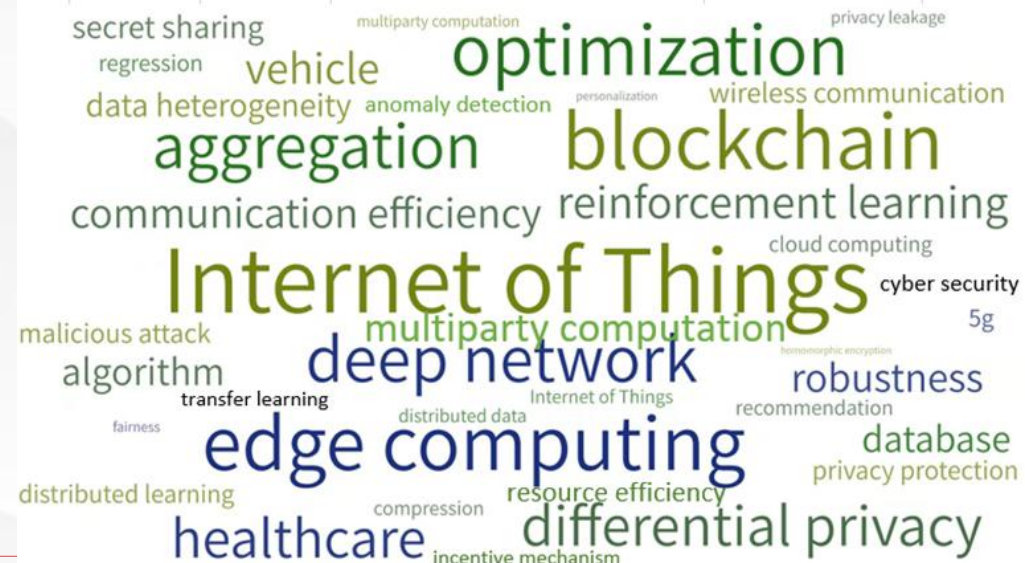
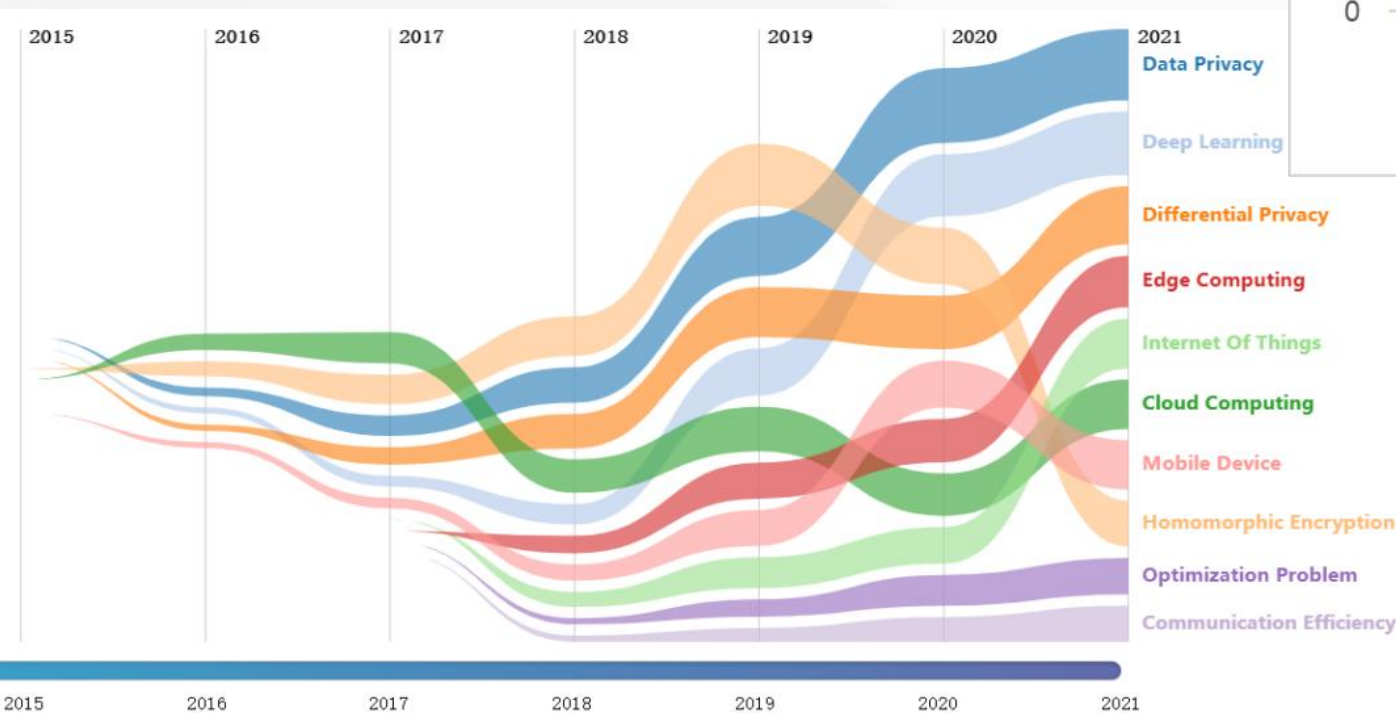
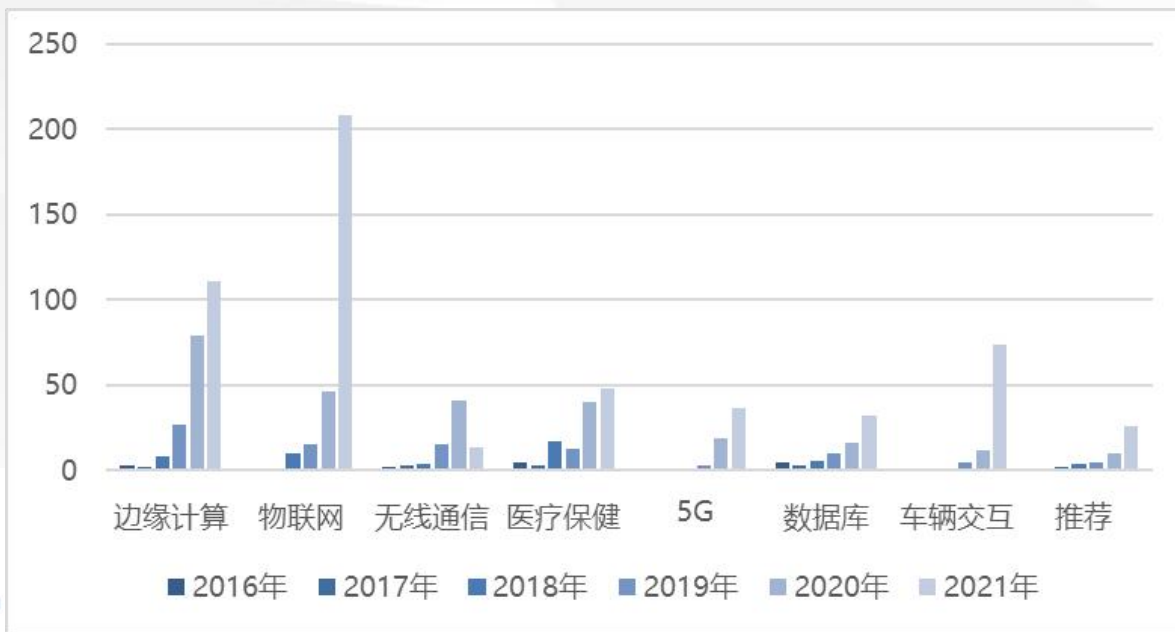
- ④ 联邦学习相关国家自然科学基金项目获批数呈现出逐年增加的趋势，这表明联邦学习在学术界和工业界的应用越来越受到重视。联邦学习是多个领域的融合，触及到的研究方向众多，应用场景非常丰富。
- ④ 金融领域有复旦大学柴洪峰团队获批的重大研究计划《大数据背景下基于联邦学习的小微企业信用风险评估研究》
- ④ 通信领域有深圳市大数据研究院朱光旭团队获批的青年科学基金项目《面向联邦式边缘学习的高效通信技术研究》
- ④ 医疗领域有上海大学武星团队的面上项目《多中心胶囊内窥镜影像联邦主动学习》





① 涵盖多个领域，主要以边缘计算、物联网和车辆交互为主

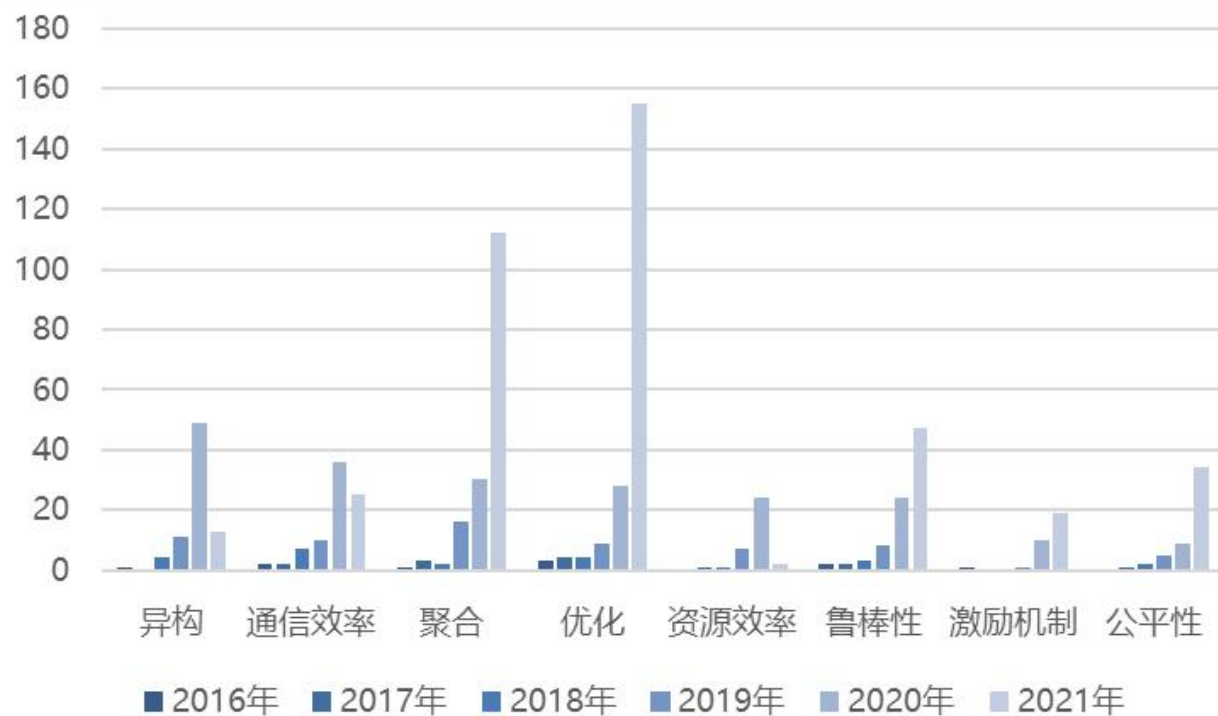
② 各领域联邦学习研究逐年增长，趋势良好





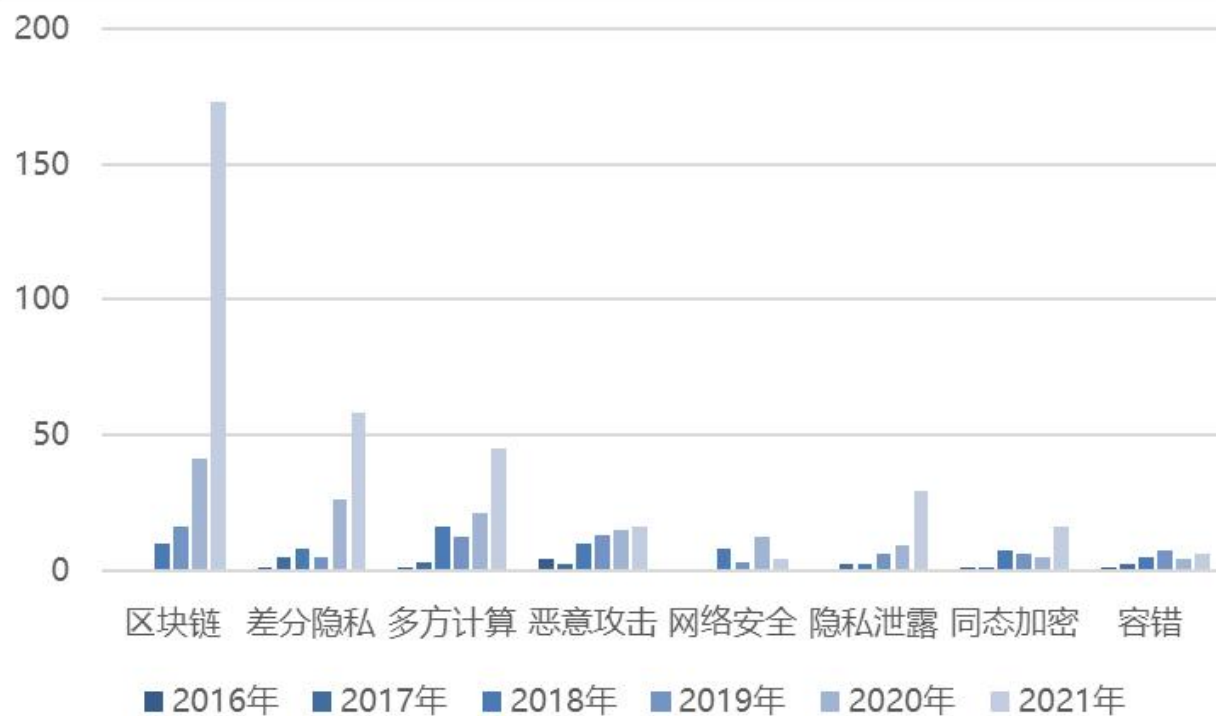
联邦学习性能上以聚合、优化方法和异构为主

各方法研究量逐年增长



联邦学习安全上以区块链、差分隐私和多方计算为主

各方法研究量逐年增长





开源框架

Github热度	发布方	系统名称	开源时间	系统特点
8000	OpenMind	PySyft	2017.7	• 一个用于安全和私有深度学习的 Python 库 • 基于 PyTorch, 使用 Unity Game Engine • 安全多方计算 • 联合学习、差异隐私
4100	微众银行	FATE	2019.2	• 工业级框架, 采用 Python开发, 底层计算存储基于 EGGROLL、Spark 等高性能计算引擎 • 提供一站式的联邦模型企业级服务解决方案。提供多插件支持联邦学习企业和科研应用 • 支持主流的分类、回归、聚类和迁移学习的联邦化算法 • 提供多种安全计算协议支撑上层应用, 支持同态加密协议、秘密共享协议、不经意传输协议和 DH密钥交换算法等 • 提供20多个联邦算法组件
1800	谷歌	TensorFlow Federated	2019.3	• 可以选择 ML 模型架构 • 模型设计理念以数据为主

04

FedAvg Algorithm





Non-IID

The training data on a given client is typically based on the usage of the mobile device by a particular user, and hence any particular user's local dataset will not be representative of the population distribution.

Unbalanced

Similarly, some users will make much heavier use of the service or app than others, leading to varying amounts of local training data.

Massively distributed

We expect the number of clients participating in an optimization to be much larger than the average number of examples per client.

Limited communication

Mobile devices are frequently offline or on slow or expensive connections.



$$\min_{w \in \mathbb{R}^d} f(w) \quad \text{where} \quad f(w) \stackrel{\text{def}}{=} \frac{1}{n} \sum_{i=1}^n f_i(w)$$

$f_i(w) = l(x_i; y_i; w)$, that is, the loss of the prediction on example $(x_i; y_i)$ made with model parameters w .

We assume there are K clients over which the data is partitioned, with $\mathcal{P}_k$ the set of indexes of data points on client k . Thus, we can re-write the objective

$$f(w) = \sum_{k=1}^K \frac{n_k}{n} F_k(w) \quad \text{where} \quad F_k(w) = \frac{1}{n_k} \sum_{i \in \mathcal{P}_k} f_i(w)$$



From SGD to FedAvg

$$F_k(w) = \frac{1}{n_k} \sum_{i \in \mathcal{P}_k} f_i(w)$$

$$g_k = \nabla F_k(w_t)$$

$$w_{t+1} \leftarrow w_t - \eta \sum_{k=1}^K \frac{n_k}{n} g_k$$

$$\forall k, w_{t+1}^k \leftarrow w_t - \eta g_k$$

$$w_{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k$$

Algorithm 1 FederatedAveraging. The K clients are indexed by k ; B is the local minibatch size, E is the number of local epochs, and η is the learning rate.

Server executes:

initialize w_0

for each round $t = 1, 2, \dots$ **do**

$m \leftarrow \max(C \cdot K, 1)$

$S_t \leftarrow$ (random set of m clients)

for each client $k \in S_t$ **in parallel do**

$w_{t+1}^k \leftarrow \text{ClientUpdate}(k, w_t)$

$w_{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k$

ClientUpdate(k, w): *// Run on client k*

$\mathcal{B} \leftarrow$ (split $\mathcal{P}_k$ into batches of size B)

for each local epoch i from 1 to E **do**

for batch $b \in \mathcal{B}$ **do**

$w \leftarrow w - \eta \nabla \ell(w; b)$

 return w to server





FedAvg algorithm

Experiments

- Image classification: MNIST handwritten digital recognition
- Language modeling: Dataset based on the Complete Works of William Shakespeare



⊗ Experiments (MNIST)

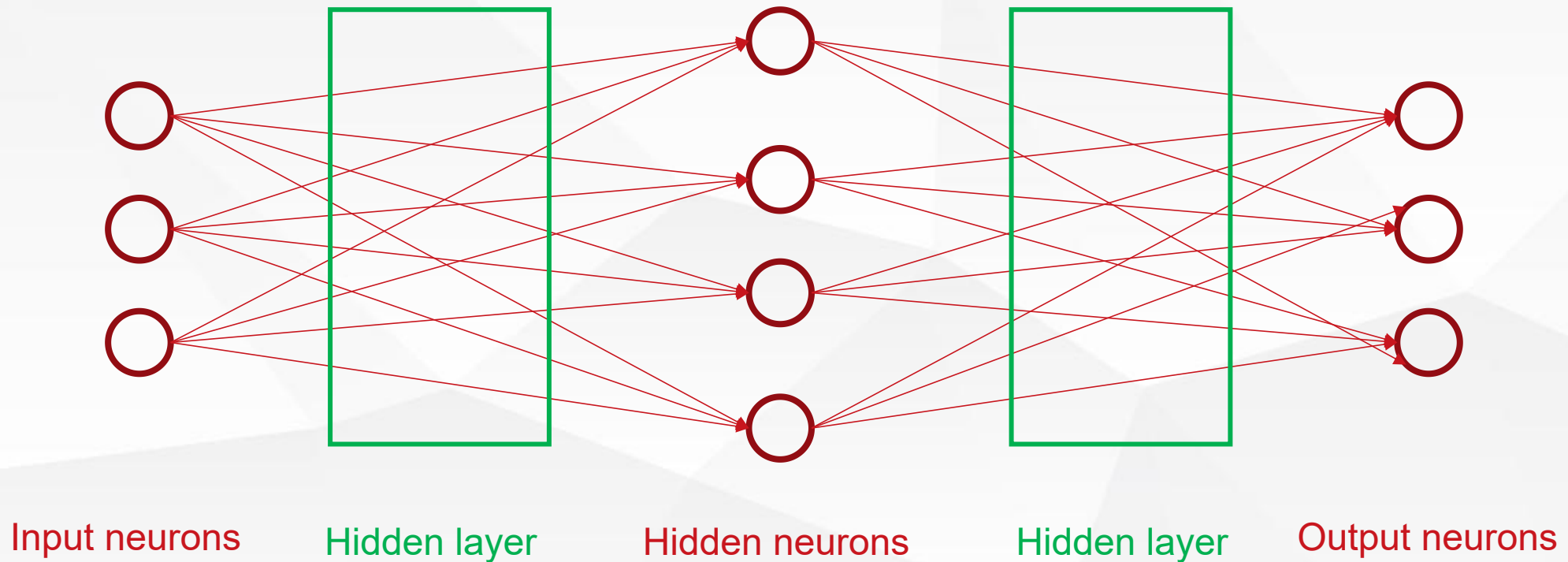
- IID: shuffling the data and dividing it into 100 clients, each receiving 600 examples.
- Non-IID: (1) sort the data by number label; (2) divide it into 200 shards of size 300; (3) assign 2 shards to each of the 100 clients, most clients will have only two number examples.



FedAvg algorithm

Experiments (MNIST)

- 1) A simple multilayer-perceptron with 2-hidden layers with 200 units each using ReLU activations (199,210 total parameters), which we refer to as the MNIST 2NN.





Experiments (MNIST)

- 1) A simple multilayer-perceptron with 2-hidden layers with 200 units each using ReLU activations (199,210 total parameters), which we refer to as the MNIST 2NN.

```
class FedAvgMLP(nn.Module):
    def __init__(self, in_features=784, num_classes=10, hidden_dim=200):
        super().__init__()
        self.fc1 = nn.Linear(in_features, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, num_classes)
        self.act = nn.ReLU(inplace=True)

    def forward(self, x):
        if x.ndim == 4:
            x = x.view(x.size(0), -1)
        x = self.act(self.fc1(x))
        x = self.fc2(x)
        return x
```





Experiments (MNIST)

- 2) A CNN with two 5x5 convolution layers (the first with 32 channels, the second with 64, each followed with 2x2 max pooling), a fully connected layer with 512 units and ReLU activation, and a final softmax output layer (1,663,370 total parameters).

```
class FedAvgCNN(nn.Module):
    def __init__(self, in_features=1, num_classes=10, dim=1024):
        super().__init__()
        self.conv1 = nn.Sequential(
            nn.Conv2d(in_features,
                      32,
                      kernel_size=5,
                      padding=0,
                      stride=1,
                      bias=True),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=(2, 2))
        )
```





FedAvg algorithm

Experiments (Key hyperparameters)

- C: Proportion of clients participating in calculation: 1 indicates that all clients participate in training
- E: Number of training cycles per client between two communications
- B: Mini-batch size of each client. ∞ indicates full-batch
- $u_k = E \frac{n_k}{B}$: The total number of updates in each iteration on client k



FedAvg algorithm



Experiments (communication rounds)

2NN <i>C</i>	IID		Non-IID	
	$B = \infty$	$B = 10$	$B = \infty$	$B = 10$
0.0	1455	316	4278	3275
0.1	1474 (1.0×)	87 (3.6×)	1796 (2.4×)	664 (4.9×)
0.2	1658 (0.9×)	77 (4.1×)	1528 (2.8×)	619 (5.3×)
0.5	— (—)	75 (4.2×)	— (—)	443 (7.4×)
1.0	— (—)	70 (4.5×)	— (—)	380 (8.6×)

CNN, $E = 5$				
0.0	387	50	1181	956
0.1	339 (1.1×)	18 (2.8×)	1100 (1.1×)	206 (4.6×)
0.2	337 (1.1×)	18 (2.8×)	978 (1.2×)	200 (4.8×)
0.5	164 (2.4×)	18 (2.8×)	1067 (1.1×)	261 (3.7×)
1.0	246 (1.6×)	16 (3.1×)	— (—)	97 (9.9×)

C = 0.1 is the best

C: Proportion of clients participating in calculation

E: Number of training cycles per client between two communications

B: Mini-batch size of each client. ∞ indicates full-batch

$u_k = E \frac{n_k}{B}$: The total number of updates in each iteration on client k





FedAvg algorithm

Experiments (communication rounds)

MNIST CNN, 99% ACCURACY					
CNN	E	B	u	IID	Non-IID
FEDSGD	1	∞	1	626	483
FEDAVG	5	∞	5	179 (3.5 $\times$)	1000 (0.5 $\times$)
FEDAVG	1	50	12	65 (9.6 $\times$)	600 (0.8 $\times$)
FEDAVG	20	∞	20	234 (2.7 $\times$)	672 (0.7 $\times$)
FEDAVG	1	10	60	34 (18.4 $\times$)	350 (1.4 $\times$)
FEDAVG	5	50	60	29 (21.6 $\times$)	334 (1.4 $\times$)
FEDAVG	20	50	240	32 (19.6 $\times$)	426 (1.1 $\times$)
FEDAVG	5	10	300	20 (31.3 $\times$)	229 (2.1 $\times$)
FEDAVG	20	10	1200	18 (34.8 $\times$)	173 (2.8 $\times$)

SHAKESPEARE LSTM, 54% ACCURACY					
LSTM	E	B	u	IID	Non-IID
FEDSGD	1	∞	1.0	2488	3906
FEDAVG	1	50	1.5	1635 (1.5 $\times$)	549 (7.1 $\times$)
FEDAVG	5	∞	5.0	613 (4.1 $\times$)	597 (6.5 $\times$)
FEDAVG	1	10	7.4	460 (5.4 $\times$)	164 (23.8 $\times$)
FEDAVG	5	50	7.4	401 (6.2 $\times$)	152 (25.7 $\times$)
FEDAVG	5	10	37.1	192 (13.0 $\times$)	41 (95.3 $\times$)

FedSGD: C = 1 and Full-Batch Optimization

FedAvg: C = 0.1

C: Proportion of clients participating in calculation

E: Number of training cycles per client between two communications

B: Mini-batch size of each client. ∞ indicates full-batch

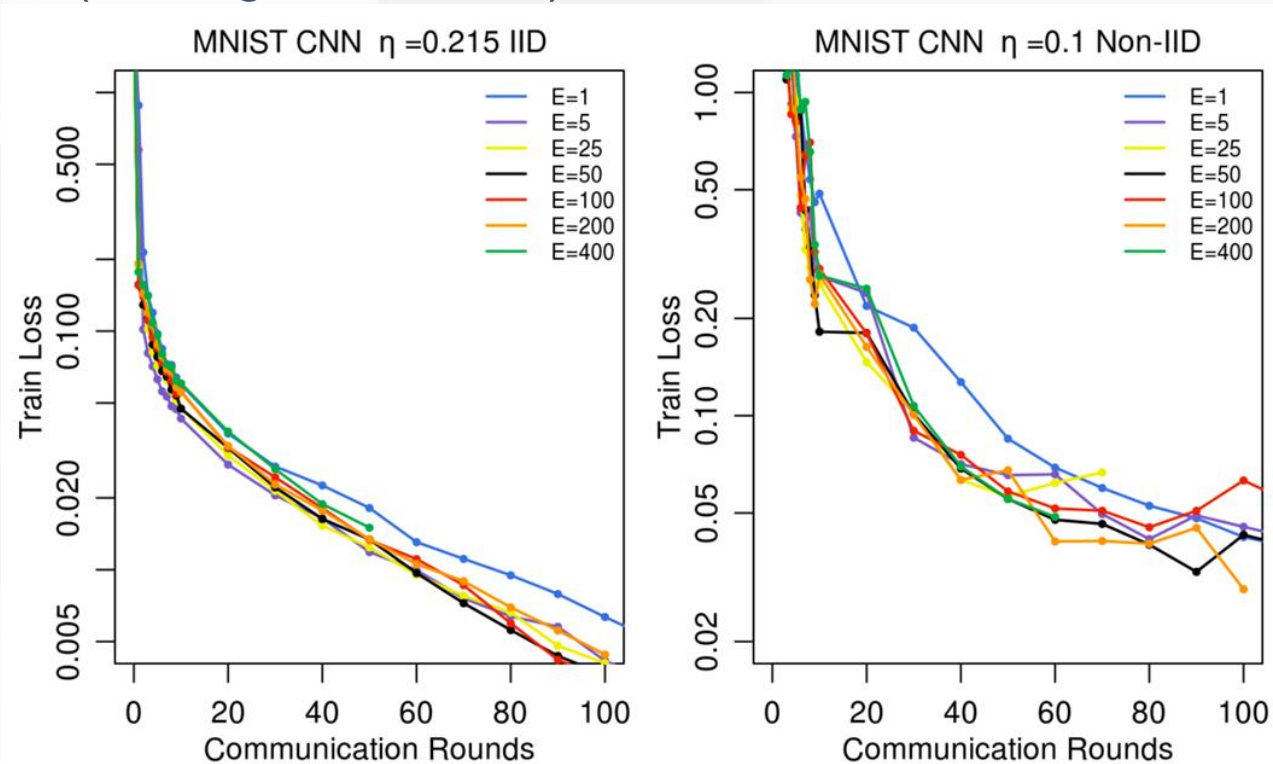
$u_k = E \frac{n_k}{B}$: The total number of updates in each iteration on client k





FedAvg algorithm

Experiments (training loss curves)



C: Proportion of clients participating in calculation, $C=0.1$

E: Number of training cycles per client between two communications

B: Mini-batch size of each client. $B=10$.

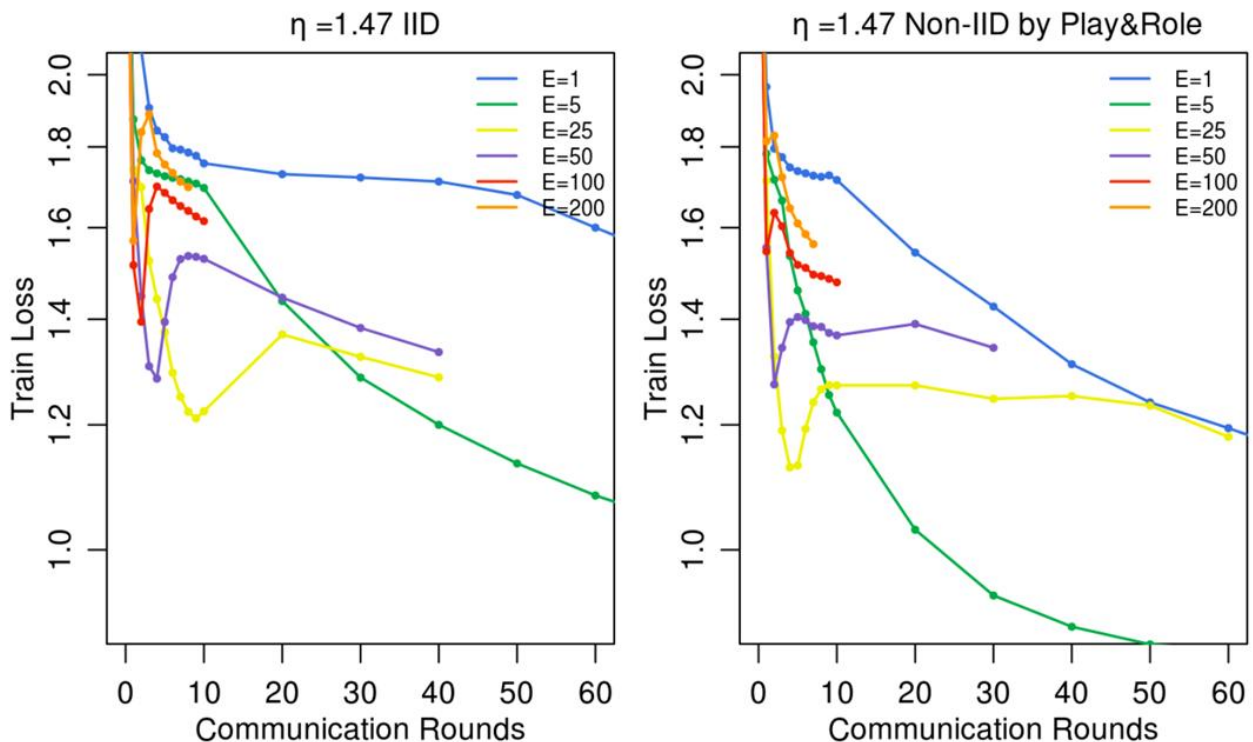
$u_k = E \frac{n_k}{B}$: The total number of updates in each iteration on client k





FedAvg algorithm

Experiments (training loss curves)



LSTM

C: Proportion of clients participating in calculation, $C=0.1$

E: Number of training cycles per client between two communications

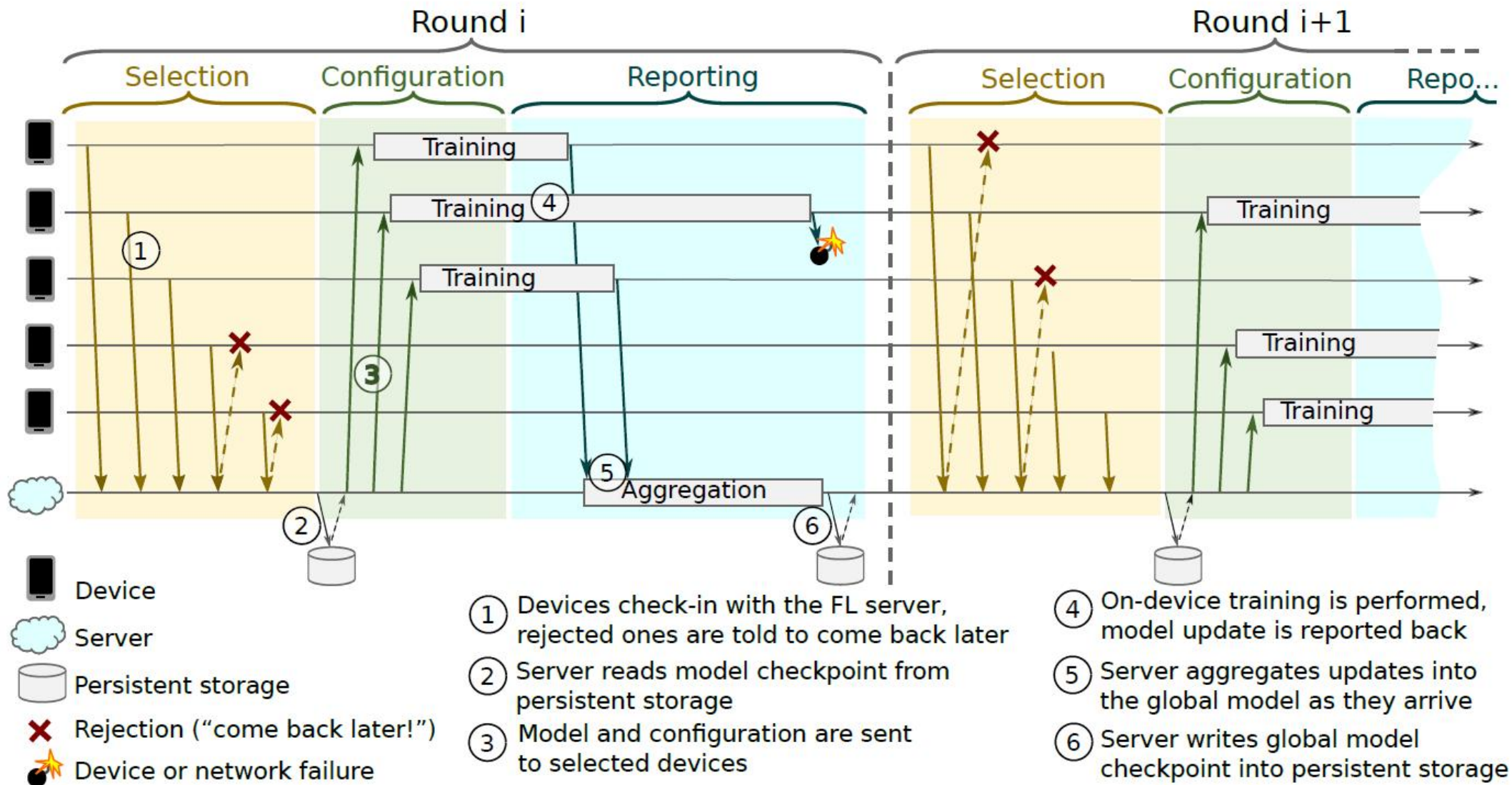
B: Mini-batch size of each client. $B=10$.

$u_k = E \frac{n_k}{B}$: The total number of updates in each iteration on client k





Federated Learning Protocol



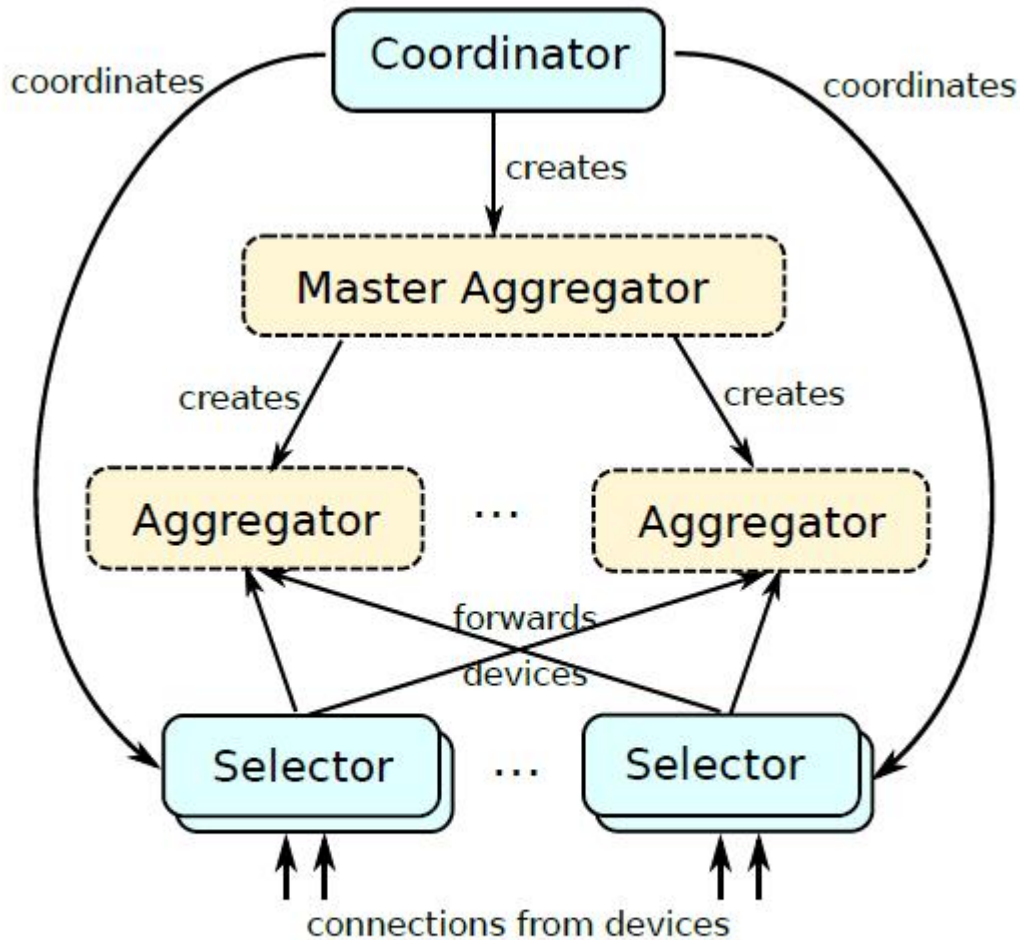


Actor Model of FL

Coordinators

The top-level actors which enable global synchronization and advancing rounds in lockstep.

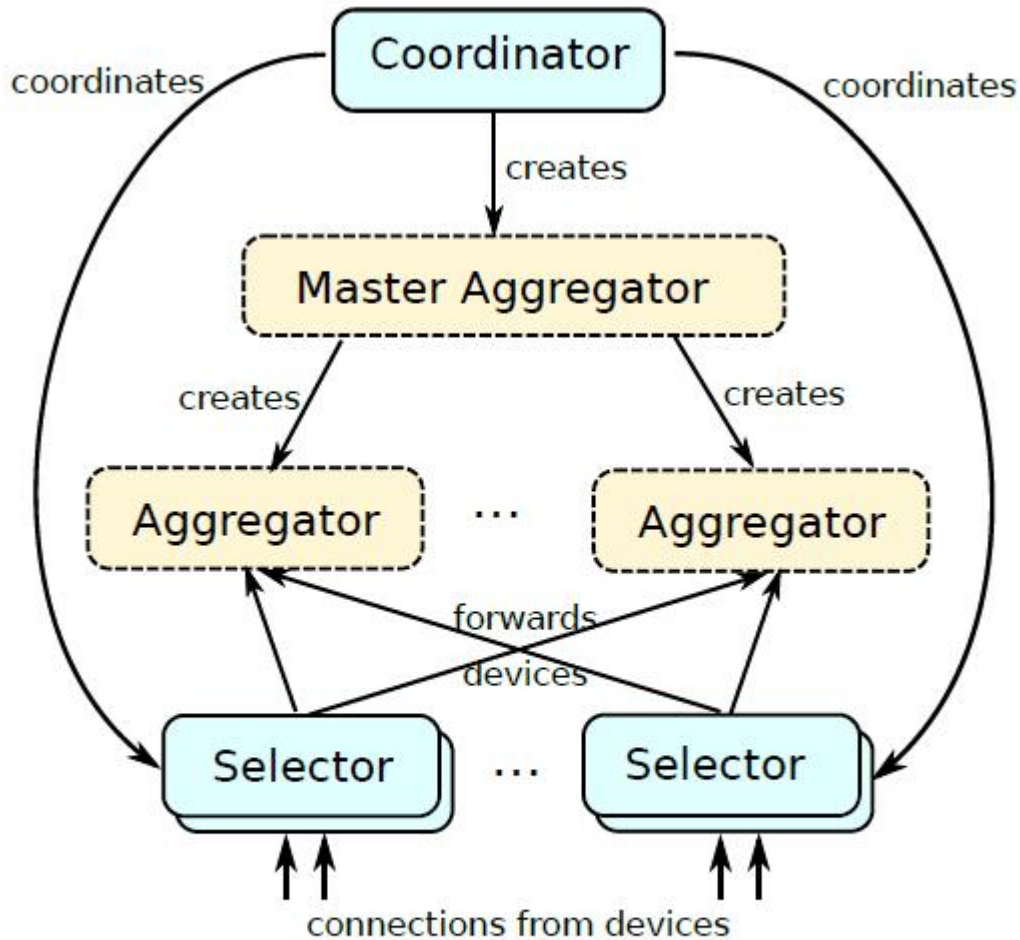
There are multiple Coordinators, and each one is responsible for an FL population of devices.



-  Persistent (long-lived) actor
-  Ephemeral (short-lived) actor



Actor Model of FL



-  Persistent (long-lived) actor
-  Ephemeral (short-lived) actor

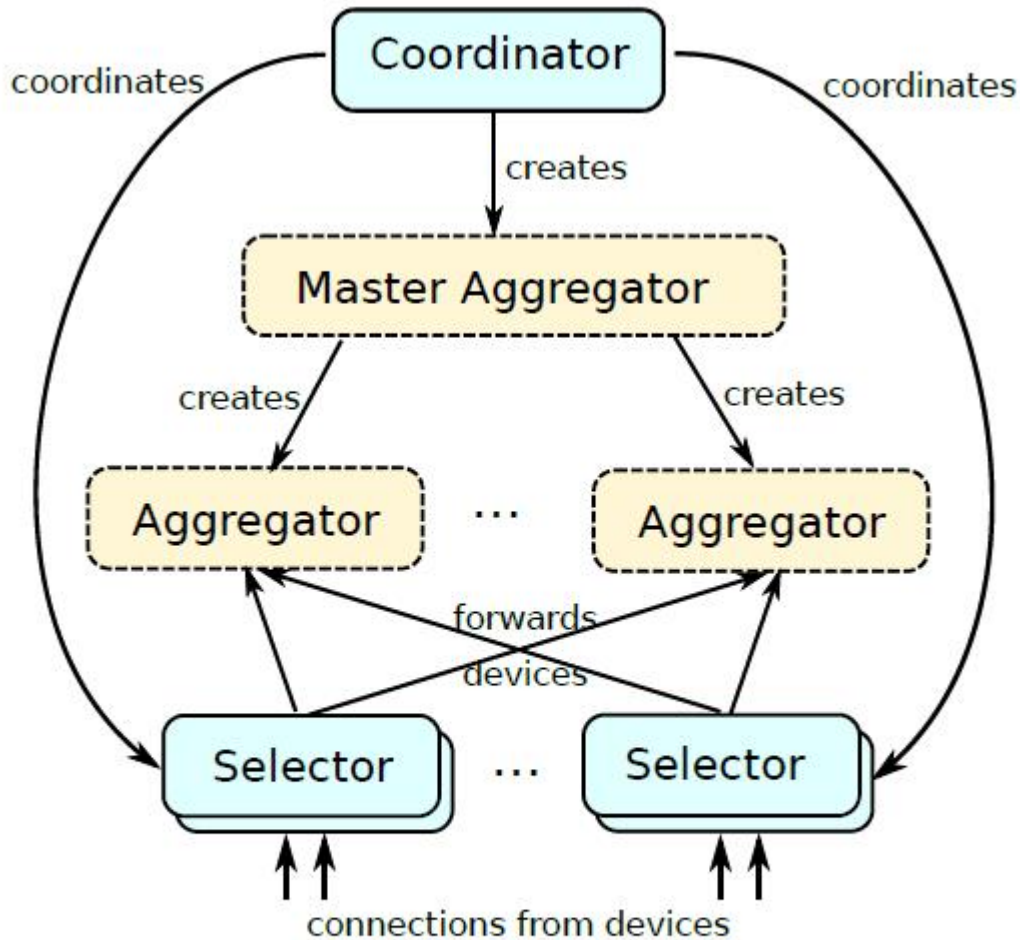
Selectors

Responsible for accepting and forwarding deviceconnections.

They periodically receive information from the Coordinator about how many devices are needed for each FL population, which they use to make local decisions about whether or not to accept each device.



Actor Model of FL



-  Persistent (long-lived) actor
-  Ephemeral (short-lived) actor

Master Aggregators

Manage the rounds of each FL task.

In order to scale with the number of devices and update size, they make dynamic decisions to spawn one or more Aggregators to which work is delegated.



上海交通大学

SHANGHAI JIAO TONG UNIVERSITY

感谢聆听

饮水思源 爱国荣校